Lattice Sigma Protocols

October 29, 2026

Blockstream Research

research.blockstream.com

github.com/BlockstreamResearch/zk-book



Plan

1. Lattice-Based Identification and Rejection Sampling
2. Proving Products of Committed Values
3. Proving Linear and Quadratic Relations over $\mathbb{Z}_q$

What will we learn today?

- How to translate the Schnorr identification protocol to lattices, and what goes wrong.

What will we learn today?

- How to translate the Schnorr identification protocol to lattices, and what goes wrong.
- **Rejection sampling** and discrete Gaussians: the fix that makes lattice protocols zero-knowledge.

What will we learn today?

- How to translate the Schnorr identification protocol to lattices, and what goes wrong.
- **Rejection sampling** and discrete Gaussians: the fix that makes lattice protocols zero-knowledge.
- The Lyubashevsky identification protocol and signature scheme (the ancestor of ML-DSA).

What will we learn today?

- How to translate the Schnorr identification protocol to lattices, and what goes wrong.
- **Rejection sampling** and discrete Gaussians: the fix that makes lattice protocols zero-knowledge.
- The Lyubashevsky identification protocol and signature scheme (the ancestor of ML-DSA).
- BDLOP commitments and the ALS20 **product proof**; packing $\mathbb{Z}_q$ values into NTT slots.

What will we learn today?

- How to translate the Schnorr identification protocol to lattices, and what goes wrong.
- **Rejection sampling** and discrete Gaussians: the fix that makes lattice protocols zero-knowledge.
- The Lyubashevsky identification protocol and signature scheme (the ancestor of ML-DSA).
- BDLOP commitments and the ALS20 **product proof**; packing $\mathbb{Z}_q$ values into NTT slots.
- LNP22: linear and quadratic relations over $\mathbb{Z}_q$ via **constant coefficients**, and approximate range proofs.

What will we learn today?

- How to translate the Schnorr identification protocol to lattices, and what goes wrong.
- **Rejection sampling** and discrete Gaussians: the fix that makes lattice protocols zero-knowledge.
- The Lyubashevsky identification protocol and signature scheme (the ancestor of ML-DSA).
- BDLOP commitments and the ALS20 **product proof**; packing $\mathbb{Z}_q$ values into NTT slots.
- LNP22: linear and quadratic relations over $\mathbb{Z}_q$ via **constant coefficients**, and approximate range proofs.

💬 *Key idea*

One can translate the majority of existing DL-based protocols to lattice-based assumptions by employing the rejection sampling technique.

Identification and Rejection Sampling

Setting

- We work over $\mathbb{Z}_q$; ring and module versions: replace $\mathbb{Z}_q$ with $\mathcal{R}_q$.
- Short elements: $S_\eta := \{x \in \mathbb{Z}_q : \|x\|_\infty \leq \eta\}$.

Setting

- We work over $\mathbb{Z}_q$; ring and module versions: replace $\mathbb{Z}_q$ with $\mathcal{R}_q$.
- Short elements: $S_\eta := \{x \in \mathbb{Z}_q : \|x\|_\infty \leq \eta\}$.
- The **challenge set**

$$B_\omega := \{\mathbf{c} \in \{-1, 0, 1\}^k : \mathbf{c} \text{ has exactly } \omega \text{ non-zero entries}\},$$

where the weight ω is chosen so that $\#B_\omega \geq 2^\lambda$.

Setting

- We work over $\mathbb{Z}_q$; ring and module versions: replace $\mathbb{Z}_q$ with $\mathcal{R}_q$.
- Short elements: $S_\eta := \{x \in \mathbb{Z}_q : \|x\|_\infty \leq \eta\}$.
- The **challenge set**

$$B_\omega := \{\mathbf{c} \in \{-1, 0, 1\}^k : \mathbf{c} \text{ has exactly } \omega \text{ non-zero entries}\},$$

where the weight ω is chosen so that $\#B_\omega \geq 2^\lambda$.

Question #1

Verify the following two facts:

1. $\#B_\omega = 2^\omega \binom{k}{\omega}$;
2. $\|\mathbf{Sc}\|_\infty \leq \eta\omega$ for every $\mathbf{S} \in S_\eta^{m \times k}$ and $\mathbf{c} \in B_\omega$.

Setting

- We work over $\mathbb{Z}_q$; ring and module versions: replace $\mathbb{Z}_q$ with $\mathcal{R}_q$.
- Short elements: $S_\eta := \{x \in \mathbb{Z}_q : \|x\|_\infty \leq \eta\}$.
- The **challenge set**

$$B_\omega := \{\mathbf{c} \in \{-1, 0, 1\}^k : \mathbf{c} \text{ has exactly } \omega \text{ non-zero entries}\},$$

where the weight ω is chosen so that $\#B_\omega \geq 2^\lambda$.

Question #1

Verify the following two facts:

1. $\#B_\omega = 2^\omega \binom{k}{\omega}$;
2. $\|\mathbf{Sc}\|_\infty \leq \eta\omega$ for every $\mathbf{S} \in S_\eta^{m \times k}$ and $\mathbf{c} \in B_\omega$.

Remark

Changing today's Gaussian distribution to the uniform one almost immediately gives Dilithium (ML-DSA).

Recall: Schnorr Identification

The prover holds $\alpha \in \mathbb{Z}_q$, the verifier holds $h = g^\alpha$ for $\mathbb{G} = \langle g \rangle$ of prime order q.

KeyGen(1^λ):

1: $\alpha \leftarrow\!\!\$\ \mathbb{Z}_q$;
2: $h \leftarrow g^\alpha$;
3: **return** (h, α).

$\mathcal{P}(h \in \mathbb{G}, \alpha \in \mathbb{Z}_q)$:

1: $r \leftarrow\!\!\$\ \mathbb{Z}_q$;
2: **send** $w \leftarrow g^r$;
3: **receive** c;
4: **send** $z \leftarrow r + \alpha c$.

$\mathcal{V}(h \in \mathbb{G}) \to \{0, 1\}$:

1: **receive** w;
2: **send** $c \leftarrow\!\!\$\ \mathbb{Z}_q$;
3: **receive** z;
4: **return** $[\![g^z = wh^c]\!]$.

Recall: Schnorr Identification

The prover holds $\alpha \in \mathbb{Z}_q$, the verifier holds $h = g^\alpha$ for $\mathbb{G} = \langle g \rangle$ of prime order q.

KeyGen(1^λ):
1: $\alpha \leftarrow\!\!\$\ \mathbb{Z}_q$;
2: $h \leftarrow g^\alpha$;
3: **return** (h, α).

$\mathcal{P}(h \in \mathbb{G}, \alpha \in \mathbb{Z}_q)$:
1: $r \leftarrow\!\!\$\ \mathbb{Z}_q$;
2: **send** $w \leftarrow g^r$;
3: **receive** c;
4: **send** $z \leftarrow r + \alpha c$.

$\mathcal{V}(h \in \mathbb{G}) \to \{0, 1\}$:
1: **receive** w;
2: **send** $c \leftarrow\!\!\$\ \mathbb{Z}_q$;
3: **receive** z;
4: **return** $[\![g^z = wh^c]\!]$.

Two properties make this protocol secure:

- **Special soundness.** $\alpha \mapsto g^\alpha$ is one-way, and from (w, c, z), (w, c', z') with $c \neq c'$ one extracts $\alpha = (z - z')/(c - c')$.

Recall: Schnorr Identification

The prover holds $\alpha \in \mathbb{Z}_q$, the verifier holds $h = g^\alpha$ for $\mathbb{G} = \langle g \rangle$ of prime order q.

KeyGen(1^λ):
1: $\alpha \leftarrow\!\!\$\ \mathbb{Z}_q$;
2: $h \leftarrow g^\alpha$;
3: **return** (h, α).

$\mathcal{P}(h \in \mathbb{G}, \alpha \in \mathbb{Z}_q)$:
1: $r \leftarrow\!\!\$\ \mathbb{Z}_q$;
2: **send** $w \leftarrow g^r$;
3: **receive** c;
4: **send** $z \leftarrow r + \alpha c$.

$\mathcal{V}(h \in \mathbb{G}) \to \{0, 1\}$:
1: **receive** w;
2: **send** $c \leftarrow\!\!\$\ \mathbb{Z}_q$;
3: **receive** z;
4: **return** $[\![g^z = wh^c]\!]$.

Two properties make this protocol secure:

- **Special soundness.** $\alpha \mapsto g^\alpha$ is one-way, and from (w, c, z), (w, c', z') with $c \neq c'$ one extracts $\alpha = (z - z')/(c - c')$.
- **Zero-knowledge.** $z = r + \alpha c$ is uniform in $\mathbb{Z}_q$ for every fixed α.

Recall: Schnorr Identification

The prover holds $\alpha \in \mathbb{Z}_q$, the verifier holds $h = g^\alpha$ for $\mathbb{G} = \langle g \rangle$ of prime order q.

KeyGen(1^λ):
1: $\alpha \leftarrow\!\!\$\ \mathbb{Z}_q$;
2: $h \leftarrow g^\alpha$;
3: **return** (h, α).

$\mathcal{P}(h \in \mathbb{G}, \alpha \in \mathbb{Z}_q)$:
1: $r \leftarrow\!\!\$\ \mathbb{Z}_q$;
2: **send** $w \leftarrow g^r$;
3: **receive** c;
4: **send** $z \leftarrow r + \alpha c$.

$\mathcal{V}(h \in \mathbb{G}) \to \{0, 1\}$:
1: **receive** w;
2: **send** $c \leftarrow\!\!\$\ \mathbb{Z}_q$;
3: **receive** z;
4: **return** $[\![g^z = wh^c]\!]$.

Two properties make this protocol secure:

- **Special soundness.** $\alpha \mapsto g^\alpha$ is one-way, and from (w, c, z), (w, c', z') with $c \neq c'$ one extracts $\alpha = (z - z')/(c - c')$.
- **Zero-knowledge.** $z = r + \alpha c$ is uniform in $\mathbb{Z}_q$ for every fixed α.

We now try to preserve both properties in the lattice setting.

Attempt #1: Replace Exponentiation with a Matrix

Candidate for $\alpha \mapsto g^\alpha$: the linear map $\mathbf{S} \mapsto \mathbf{AS}$ for a public random $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$. Changes in orange.

KeyGen(1^λ):

1: $\mathbf{S} \leftarrow\!\$\ \mathbb{Z}_q^{m \times k}$;
2: $\mathbf{T} \leftarrow \mathbf{AS}$;
3: **return** $(\mathbf{T}, \mathbf{S})$.

$\mathcal{P}(\mathbf{T}, \mathbf{S})$:

1: $\mathbf{r} \leftarrow\!\$\ \mathbb{Z}_q^m$;
2: **send** $\mathbf{w} \leftarrow \mathbf{Ar}$;
3: **receive** $\mathbf{c}$;
4: **send** $\mathbf{z} \leftarrow \mathbf{r} + \mathbf{Sc}$.

$\mathcal{V}(\mathbf{T}) \to \{0, 1\}$:

1: **receive** $\mathbf{w}$;
2: **send** $\mathbf{c} \leftarrow\!\$\ \mathbb{Z}_q^k$;
3: **receive** $\mathbf{z}$;
4: **return** $[\![\mathbf{Az} = \mathbf{w} + \mathbf{Tc}]\!]$.

Attempt #1: Replace Exponentiation with a Matrix

Candidate for $\alpha \mapsto g^\alpha$: the linear map $\mathbf{S} \mapsto \mathbf{AS}$ for a public random $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$. Changes in orange.

KeyGen(1^λ):

1: $\mathbf{S} \leftarrow\$ \mathbb{Z}_q^{m \times k}$;
2: $\mathbf{T} \leftarrow \mathbf{AS}$;
3: **return** $(\mathbf{T}, \mathbf{S})$.

$\mathcal{P}(\mathbf{T}, \mathbf{S})$:

1: $\mathbf{r} \leftarrow\$ \mathbb{Z}_q^m$;
2: **send** $\mathbf{w} \leftarrow \mathbf{Ar}$;
3: **receive** $\mathbf{c}$;
4: **send** $\mathbf{z} \leftarrow \mathbf{r} + \mathbf{Sc}$.

$\mathcal{V}(\mathbf{T}) \to \{0, 1\}$:

1: **receive** $\mathbf{w}$;
2: **send** $\mathbf{c} \leftarrow\$ \mathbb{Z}_q^k$;
3: **receive** $\mathbf{z}$;
4: **return** $[\![\mathbf{Az} = \mathbf{w} + \mathbf{Tc}]\!]$.

- **Correct**: $\mathbf{Az} = \mathbf{Ar} + \mathbf{ASc} = \mathbf{w} + \mathbf{Tc}$. ✓
- **Zero-knowledge**: $\mathbf{z}$ is uniform in $\mathbb{Z}_q^m$, exactly as in Schnorr. ✓

Attempt #1: Replace Exponentiation with a Matrix

Candidate for $\alpha \mapsto g^\alpha$: the linear map $\mathbf{S} \mapsto \mathbf{AS}$ for a public random $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$. Changes in orange.

KeyGen(1^λ):

1: $\mathbf{S} \leftarrow\$ \mathbb{Z}_q^{m \times k}$;
2: $\mathbf{T} \leftarrow \mathbf{AS}$;
3: **return** $(\mathbf{T}, \mathbf{S})$.

$\mathcal{P}(\mathbf{T}, \mathbf{S})$:

1: $\mathbf{r} \leftarrow\$ \mathbb{Z}_q^m$;
2: **send** $\mathbf{w} \leftarrow \mathbf{Ar}$;
3: **receive** $\mathbf{c}$;
4: **send** $\mathbf{z} \leftarrow \mathbf{r} + \mathbf{Sc}$.

$\mathcal{V}(\mathbf{T}) \to \{0, 1\}$:

1: **receive** $\mathbf{w}$;
2: **send** $\mathbf{c} \leftarrow\$ \mathbb{Z}_q^k$;
3: **receive** $\mathbf{z}$;
4: **return** $[\![\mathbf{Az} = \mathbf{w} + \mathbf{Tc}]\!]$.

- **Correct**: $\mathbf{Az} = \mathbf{Ar} + \mathbf{ASc} = \mathbf{w} + \mathbf{Tc}$. ✓
- **Zero-knowledge**: $\mathbf{z}$ is uniform in $\mathbb{Z}_q^m$, exactly as in Schnorr. ✓

Question #2

Why is this protocol completely unsound?

Attempt #1: Replace Exponentiation with a Matrix

Candidate for $\alpha \mapsto g^\alpha$: the linear map $\mathbf{S} \mapsto \mathbf{AS}$ for a public random $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$. Changes in orange.

KeyGen(1^λ):
1: $\mathbf{S} \leftarrow\$ \mathbb{Z}_q^{m \times k}$;
2: $\mathbf{T} \leftarrow \mathbf{AS}$;
3: **return** $(\mathbf{T}, \mathbf{S})$.

$\mathcal{P}(\mathbf{T}, \mathbf{S})$:
1: $\mathbf{r} \leftarrow\$ \mathbb{Z}_q^m$;
2: **send** $\mathbf{w} \leftarrow \mathbf{Ar}$;
3: **receive** $\mathbf{c}$;
4: **send** $\mathbf{z} \leftarrow \mathbf{r} + \mathbf{Sc}$.

$\mathcal{V}(\mathbf{T}) \to \{0, 1\}$:
1: **receive** $\mathbf{w}$;
2: **send** $\mathbf{c} \leftarrow\$ \mathbb{Z}_q^k$;
3: **receive** $\mathbf{z}$;
4: **return** $[\![\mathbf{Az} = \mathbf{w} + \mathbf{Tc}]\!]$.

- **Correct**: $\mathbf{Az} = \mathbf{Ar} + \mathbf{ASc} = \mathbf{w} + \mathbf{Tc}$. ✓
- **Zero-knowledge**: $\mathbf{z}$ is uniform in $\mathbb{Z}_q^m$, exactly as in Schnorr. ✓

Question #2

Why is this protocol completely unsound?

Since $m > n$, Gaussian elimination finds *some* $\mathbf{S}'$ with $\mathbf{AS}' = \mathbf{T}$. $\mathbf{S} \mapsto \mathbf{AS}$ is one-way only on *short* inputs, by (I)SIS! ✗

Attempt #2: Make Everything Short

Make the secret short and let the verifier check that $\mathbf{z}$ is short. For an honest $\mathbf{z} = \mathbf{r} + \mathbf{Sc}$ to pass, both terms must be short.

KeyGen(1^λ):

1: $\mathbf{S} \leftarrow\$ S_\eta^{m \times k}$;
2: $\mathbf{T} \leftarrow \mathbf{AS}$;
3: **return** $(\mathbf{T}, \mathbf{S})$.

$\mathcal{P}(\mathbf{T}, \mathbf{S})$:

1: $\mathbf{r} \leftarrow\$ \chi$;
2: **send** $\mathbf{w} \leftarrow \mathbf{Ar}$;
3: **receive** $\mathbf{c}$;
4: **send** $\mathbf{z} \leftarrow \mathbf{r} + \mathbf{Sc}$ over $\mathbb{Z}$.

$\mathcal{V}(\mathbf{T}) \rightarrow \{0, 1\}$:

1: **receive** $\mathbf{w}$;
2: **send** $\mathbf{c} \leftarrow\$ B_\omega$;
3: **receive** $\mathbf{z}$;
4: **assert** $\|\mathbf{z}\|_2 \leq \beta$;
5: **return** $[\![\mathbf{Az} = \mathbf{w} + \mathbf{Tc}]\!]$.

Here, χ is a short distribution over $\mathbb{Z}^m$, so $\|\mathbf{Sc}\|_\infty \leq \eta\omega$ and $\mathbf{r}$ are both short.

Attempt #2: Make Everything Short

Make the secret short and let the verifier check that $\mathbf{z}$ is short. For an honest $\mathbf{z} = \mathbf{r} + \mathbf{Sc}$ to pass, both terms must be short.

KeyGen(1^λ):

1: $\mathbf{S} \leftarrow\$ S_\eta^{m \times k}$;
2: $\mathbf{T} \leftarrow \mathbf{AS}$;
3: **return** $(\mathbf{T}, \mathbf{S})$.

$\mathcal{P}(\mathbf{T}, \mathbf{S})$:

1: $\mathbf{r} \leftarrow\$ \chi$;
2: **send** $\mathbf{w} \leftarrow \mathbf{Ar}$;
3: **receive** $\mathbf{c}$;
4: **send** $\mathbf{z} \leftarrow \mathbf{r} + \mathbf{Sc}$ over $\mathbb{Z}$.

$\mathcal{V}(\mathbf{T}) \rightarrow \{0, 1\}$:

1: **receive** $\mathbf{w}$;
2: **send** $\mathbf{c} \leftarrow\$ B_\omega$;
3: **receive** $\mathbf{z}$;
4: **assert** $\|\mathbf{z}\|_2 \leq \beta$;
5: **return** $[\![\mathbf{Az} = \mathbf{w} + \mathbf{Tc}]\!]$.

Here, χ is a short distribution over $\mathbb{Z}^m$, so $\|\mathbf{Sc}\|_\infty \leq \eta\omega$ and $\mathbf{r}$ are both short.

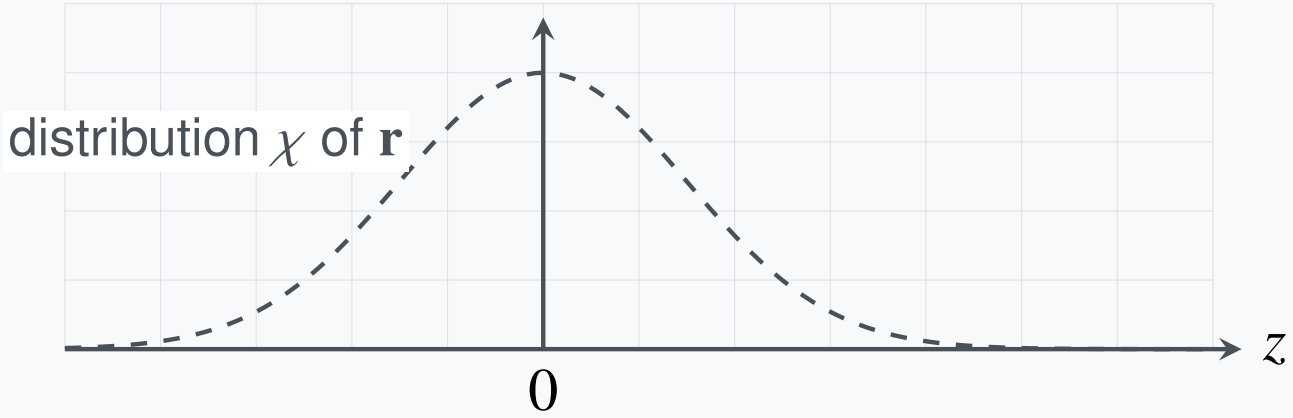
Sound! From $(\mathbf{w}, \mathbf{c}, \mathbf{z})$ and $(\mathbf{w}, \mathbf{c}', \mathbf{z}')$ with $\mathbf{c} \neq \mathbf{c}'$, substituting $\mathbf{T} = \mathbf{AS}$:

$$\mathbf{A}(\mathbf{z} - \mathbf{z}' - \mathbf{S}(\mathbf{c} - \mathbf{c}')) = \mathbf{0} \pmod{q}.$$

The vector in parentheses has norm at most $2\beta + 2\eta\omega\sqrt{m}$: unless it is zero, it is a SIS solution (same argument as Ajtai binding).

Attempt #2: the Response Leaks the Secret

A short $\mathbf{r}$ cannot hide the shift $\mathbf{Sc}$ completely: if χ is centered at zero, then $\mathbb{E}[\mathbf{z}] = \mathbf{Sc}$, and since $\mathbf{c}$ is public, enough transcripts recover $\mathbf{S}$ by linear regression.

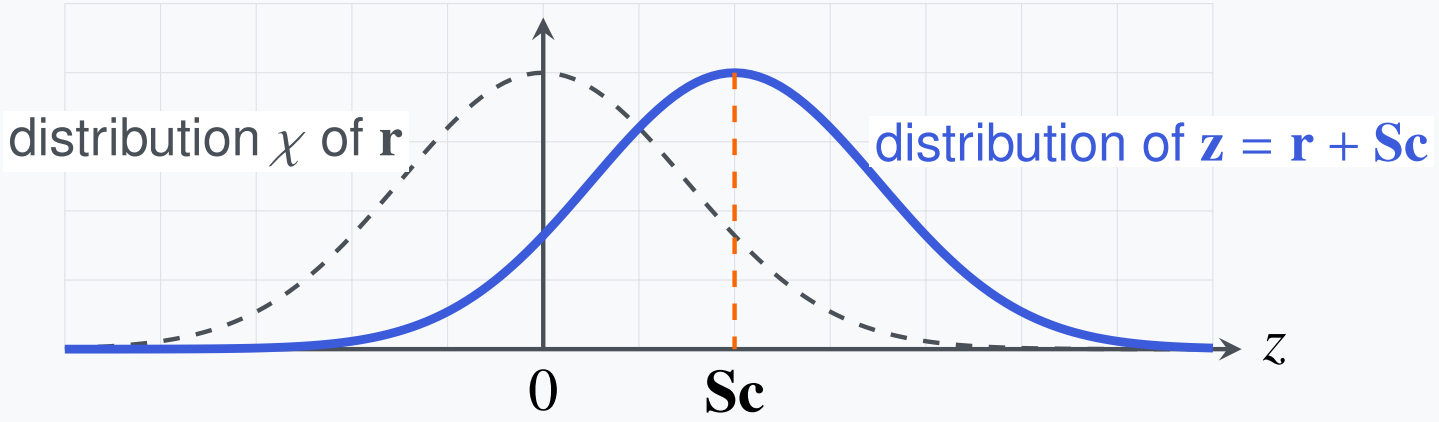


Why Schnorr does not have this problem

In Schnorr, r is uniform over the whole group $\mathbb{Z}_q$, and a uniform distribution shifted by αc is again uniform.

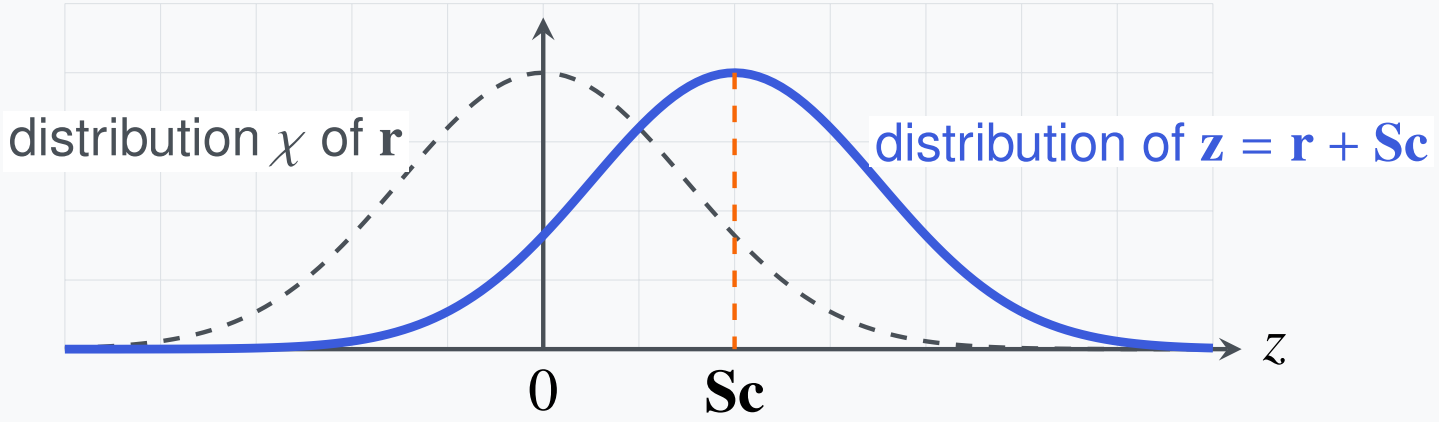
Attempt #2: the Response Leaks the Secret

A short $\mathbf{r}$ cannot hide the shift $\mathbf{Sc}$ completely: if χ is centered at zero, then $\mathbb{E}[\mathbf{z}] = \mathbf{Sc}$, and since $\mathbf{c}$ is public, enough transcripts recover $\mathbf{S}$ by linear regression.



Attempt #2: the Response Leaks the Secret

A short $\mathbf{r}$ cannot hide the shift $\mathbf{Sc}$ completely: if χ is centered at zero, then $\mathbb{E}[\mathbf{z}] = \mathbf{Sc}$, and since $\mathbf{c}$ is public, enough transcripts recover $\mathbf{S}$ by linear regression.



Why Schnorr does not have this problem

In Schnorr, r is uniform over the whole group $\mathbb{Z}_q$, and a uniform distribution shifted by αc is again uniform.

Attempt #3: Rejection Sampling

Fix: the prover *aborts* whenever $\mathbf{z}$ reveals too much about $\mathbf{S}$ and restarts with a fresh mask, so that every *sent* response follows a fixed distribution independent of $\mathbf{S}$ (**Fiat-Shamir with aborts**).

Attempt #3: Rejection Sampling

Fix: the prover *aborts* whenever $\mathbf{z}$ reveals too much about $\mathbf{S}$ and restarts with a fresh mask, so that every *sent* response follows a fixed distribution independent of $\mathbf{S}$ (**Fiat-Shamir with aborts**).

Lemma (Rejection Sampling)

Let f and g be probability distributions over a set V such that $f(z) \leq Mg(z)$ for all $z \in V$ and some constant $M \geq 1$. Consider:

1: sample $z \leftarrow\!\!\$\ g$;

2: with probability $\frac{f(z)}{Mg(z)}$, **output** z; otherwise, **goto** step 1.

The output is distributed exactly according to f, and the expected number of iterations is M.

Attempt #3: Rejection Sampling

Fix: the prover *aborts* whenever $\mathbf{z}$ reveals too much about $\mathbf{S}$ and restarts with a fresh mask, so that every *sent* response follows a fixed distribution independent of $\mathbf{S}$ (**Fiat-Shamir with aborts**).

Lemma (Rejection Sampling)

Let f and g be probability distributions over a set V such that $f(z) \leq Mg(z)$ for all $z \in V$ and some constant $M \geq 1$. Consider:

1: sample $z \leftarrow\!\!\$\ g$;

2: with probability $\frac{f(z)}{Mg(z)}$, **output** z; otherwise, **goto** step 1.

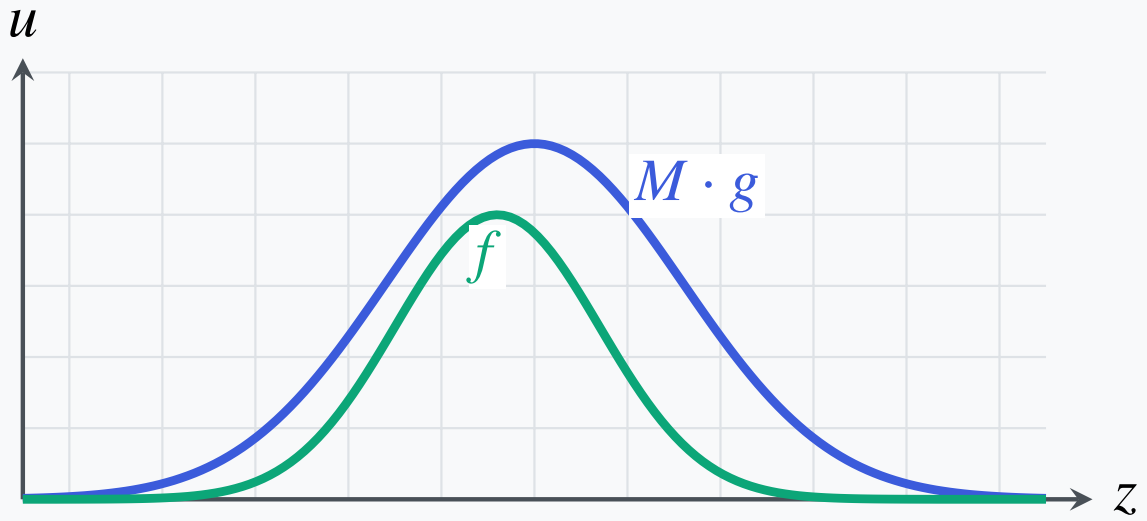
The output is distributed exactly according to f, and the expected number of iterations is M.

Proof. One iteration samples z and accepts it with probability $g(z) \cdot \frac{f(z)}{Mg(z)} = \frac{f(z)}{M}$. Summing over z: success probability $\frac{1}{M}$.

Conditioned on success, z is output with probability $\frac{f(z)/M}{1/M} = f(z)$. ■

Rejection Sampling: Picture

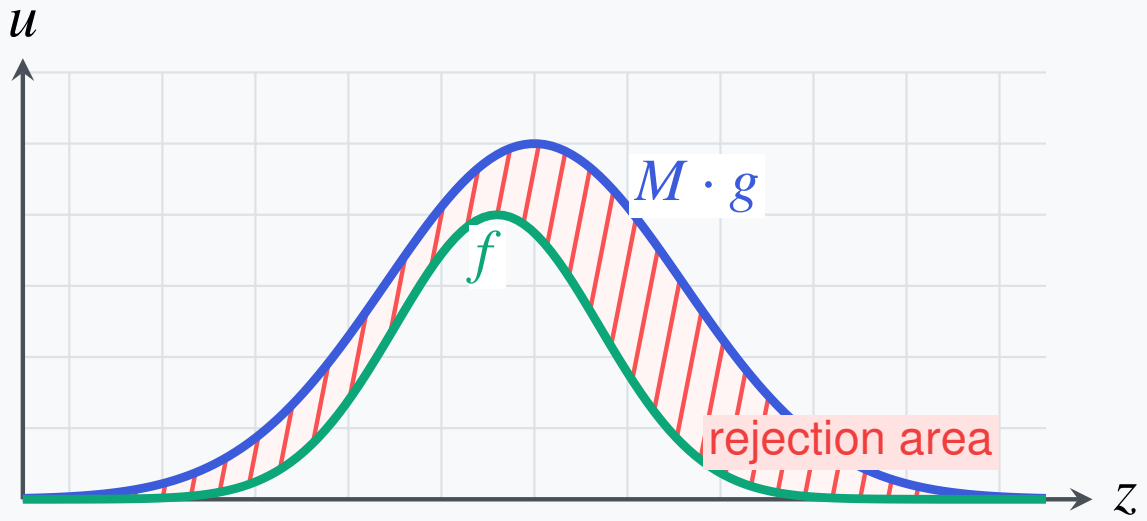
g: the *real* distribution of $\mathbf{z} = \mathbf{r} + \mathbf{Sc}$ (depends on $\mathbf{Sc}$); f: a *target* distribution independent of $\mathbf{S}$.



- $z \leftarrow\$\ g$, $u \leftarrow\$\ [0, Mg(z)]$: a uniform point (z, u) under the graph of Mg.

Rejection Sampling: Picture

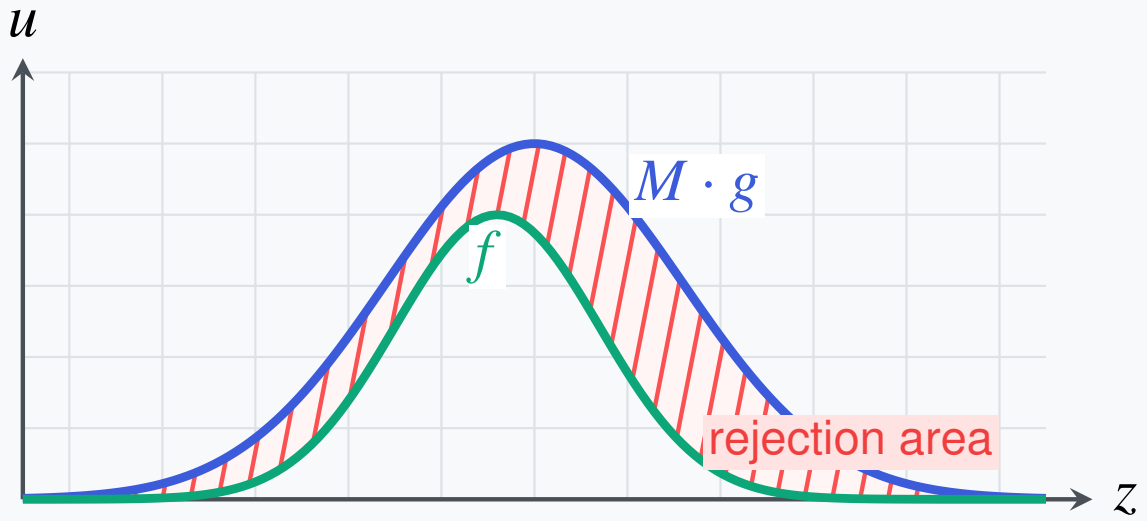
g: the *real* distribution of $\mathbf{z} = \mathbf{r} + \mathbf{Sc}$ (depends on $\mathbf{Sc}$); f: a *target* distribution independent of $\mathbf{S}$.



- $z \leftarrow\$\ g$, $u \leftarrow\$\ [0, Mg(z)]$: a uniform point (z, u) under the graph of Mg.
- Accept iff it lies under the graph of f: accepted points are uniform under f, so z follows f.

Rejection Sampling: Picture

g: the *real* distribution of $\mathbf{z} = \mathbf{r} + \mathbf{Sc}$ (depends on $\mathbf{Sc}$); f: a *target* distribution independent of $\mathbf{S}$.



- $z \leftarrow\$\ g$, $u \leftarrow\$\ [0, Mg(z)]$: a uniform point (z, u) under the graph of Mg.
- Accept iff it lies under the graph of f: accepted points are uniform under f, so z follows f.
- The closer g resembles f, the smaller M, the fewer restarts.

Discrete Gaussian Distribution

We need $f \leq Mg$ for a small M and *every* shift $\mathbf{Sc}$. The standard choice of χ:

Definition (Discrete Gaussian)

For $\sigma > 0$ and $\mathbf{v} \in \mathbb{Z}^m$, let $\rho_{\mathbf{v},\sigma}(\mathbf{x}) := \exp\left(-\|\mathbf{x} - \mathbf{v}\|_2^2 / 2\sigma^2\right)$ for $\mathbf{x} \in \mathbb{R}^m$. The **discrete Gaussian distribution** over $\mathbb{Z}^m$ with center $\mathbf{v}$ and width σ assigns to each $\mathbf{x} \in \mathbb{Z}^m$ the probability

$$\mathcal{D}^m_{\mathbf{v},\sigma}(\mathbf{x}) := \frac{\rho_{\mathbf{v},\sigma}(\mathbf{x})}{\sum_{\mathbf{y}\in\mathbb{Z}^m} \rho_\sigma(\mathbf{y})}.$$

We omit $\mathbf{v}$ when $\mathbf{v} = \mathbf{0}$: $\rho_\sigma := \rho_{\mathbf{0},\sigma}$ and $\mathcal{D}^m_\sigma := \mathcal{D}^m_{\mathbf{0},\sigma}$.

Discrete Gaussian Distribution

We need $f \leq Mg$ for a small M and *every* shift $\mathbf{Sc}$. The standard choice of χ:

Definition (Discrete Gaussian)

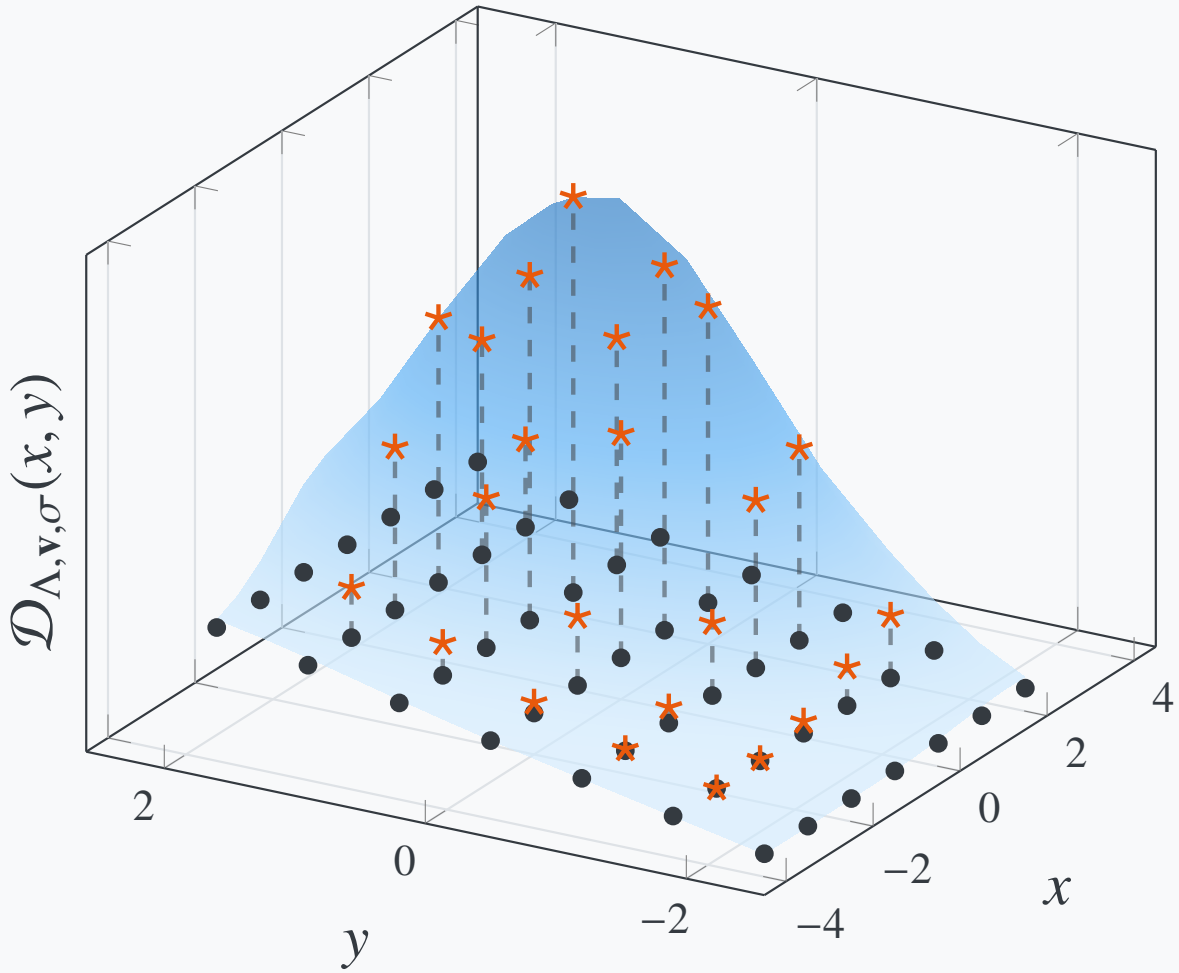
For $\sigma > 0$ and $\mathbf{v} \in \mathbb{Z}^m$, let $\rho_{\mathbf{v},\sigma}(\mathbf{x}) := \exp\left(-\|\mathbf{x} - \mathbf{v}\|_2^2 / 2\sigma^2\right)$ for $\mathbf{x} \in \mathbb{R}^m$. The **discrete Gaussian distribution** over $\mathbb{Z}^m$ with center $\mathbf{v}$ and width σ assigns to each $\mathbf{x} \in \mathbb{Z}^m$ the probability

$$\mathcal{D}^m_{\mathbf{v},\sigma}(\mathbf{x}) := \frac{\rho_{\mathbf{v},\sigma}(\mathbf{x})}{\sum_{\mathbf{y}\in\mathbb{Z}^m} \rho_\sigma(\mathbf{y})}.$$

We omit $\mathbf{v}$ when $\mathbf{v} = \mathbf{0}$: $\rho_\sigma := \rho_{\mathbf{0},\sigma}$ and $\mathcal{D}^m_\sigma := \mathcal{D}^m_{\mathbf{0},\sigma}$.

- Same normalization for every integer center (shifting permutes $\mathbb{Z}^m$).
- Some works write $\exp(-\pi\|\mathbf{x}\|_2^2/s^2)$, i.e. $s = \sqrt{2\pi}\sigma$. One can also restrict $\rho_{\mathbf{v},\sigma}$ to any lattice Λ.

Discrete Gaussian: Picture



$\mathcal{D}_{\Lambda,\mathbf{v},\sigma}$ with $\sigma = 1.2$, $\mathbf{v} = (1.2, 0.6)$ over the lattice spanned by $\mathbf{b}_1 = (1, 0)$, $\mathbf{b}_2 = (1/2, \sqrt{3}/2)$. The probability of each lattice point (stars) is proportional to the height of the continuous Gaussian above it.

Discrete Gaussian Samples are Short

Lemma (Samples are short)

For any $\tau > 1$, one has

$$\Pr_{\mathbf{z} \leftarrow\$ \mathcal{D}_\sigma^m} \left[\|\mathbf{z}\|_2 > \tau\sigma\sqrt{m} \right] < \tau^m e^{m(1-\tau^2)/2}.$$

Negligible even for $\tau = 2$, as $2^m e^{-3m/2} < 2^{-m}$: the verifier can safely reject any response with $\|\mathbf{z}\|_2 > 2\sigma\sqrt{m}$ (in practice, a threshold $\tau\sigma\sqrt{m}$ for some τ between 1.0 and 2.0).

Gaussian Rejection Sampling

Rejection sampling hides a short shift $\mathbf{v}$, and the lemma specifies the value of M.

Lemma (Gaussian rejection sampling)

Let $V \subseteq \mathbb{Z}^m$ with $\|\mathbf{v}\|_2 \leq T$ for all $\mathbf{v} \in V$, and let h be a distribution over V. Let $\sigma = \alpha T$, $M := \exp\left(\frac{12}{\alpha} + \frac{1}{2\alpha^2}\right)$. The statistical distance between $\mathcal{A}_0$ and $\mathcal{A}_1$ is at most $2^{-100}/M$:

Algorithm $\mathcal{A}_0$

1: $\mathbf{v} \hookleftarrow\$\ h$, $\mathbf{z} \hookleftarrow\$\ \mathcal{D}^m_{\mathbf{v},\sigma}$;
2: $u \hookleftarrow\$\ (0,1)$;
3: **if** $u > \frac{\mathcal{D}^m_\sigma(\mathbf{z})}{M\mathcal{D}^m_{\mathbf{v},\sigma}(\mathbf{z})}$: **goto** 1;
4: **return** $(\mathbf{z}, \mathbf{v})$.

Algorithm $\mathcal{A}_1$

1: $\mathbf{v} \hookleftarrow\$\ h$, $\mathbf{z} \hookleftarrow\$\ \mathcal{D}^m_\sigma$;
2: $u \hookleftarrow\$\ (0,1)$;
3: **if** $u > 1/M$: **goto** 1;
4: **return** $(\mathbf{z}, \mathbf{v})$.

Gaussian Rejection Sampling

Rejection sampling hides a short shift $\mathbf{v}$, and the lemma specifies the value of M.

Lemma (Gaussian rejection sampling)

Let $V \subseteq \mathbb{Z}^m$ with $\|\mathbf{v}\|_2 \leq T$ for all $\mathbf{v} \in V$, and let h be a distribution over V. Let $\sigma = \alpha T$, $M := \exp\left(\frac{12}{\alpha} + \frac{1}{2\alpha^2}\right)$. The statistical distance between $\mathcal{A}_0$ and $\mathcal{A}_1$ is at most $2^{-100}/M$:

Algorithm $\mathcal{A}_0$

1: $\mathbf{v} \hookleftarrow\$\ h$, $\mathbf{z} \hookleftarrow\$\ \mathcal{D}^m_{\mathbf{v},\sigma}$;
2: $u \hookleftarrow\$\ (0,1)$;
3: **if** $u > \frac{\mathcal{D}^m_\sigma(\mathbf{z})}{M\mathcal{D}^m_{\mathbf{v},\sigma}(\mathbf{z})}$: **goto** 1;
4: **return** $(\mathbf{z}, \mathbf{v})$.

Algorithm $\mathcal{A}_1$

1: $\mathbf{v} \hookleftarrow\$\ h$, $\mathbf{z} \hookleftarrow\$\ \mathcal{D}^m_\sigma$;
2: $u \hookleftarrow\$\ (0,1)$;
3: **if** $u > 1/M$: **goto** 1;
4: **return** $(\mathbf{z}, \mathbf{v})$.

Moreover, $\mathcal{A}_0$ outputs something with probability at least $(1 - 2^{-100})/M$. In $\mathcal{A}_1$, $\mathbf{z}$ is independent of $\mathbf{v}$: accepted responses reveal nothing about the shift.

Why Gaussian Rejection Sampling Works

The ratio between the target and the real distribution is

$$\frac{\mathcal{D}_{\sigma}^{m}(\mathbf{z})}{\mathcal{D}_{\mathbf{v},\sigma}^{m}(\mathbf{z})} = \exp\left(\frac{\|\mathbf{v}\|_2^2 - 2\langle \mathbf{z}, \mathbf{v}\rangle}{2\sigma^2}\right).$$

- Depends on $\mathbf{z}$ only through $\langle \mathbf{z}, \mathbf{v}\rangle$: a one-dimensional problem.

Why Gaussian Rejection Sampling Works

The ratio between the target and the real distribution is

$$\frac{\mathcal{D}_{\sigma}^{m}(\mathbf{z})}{\mathcal{D}_{\mathbf{v},\sigma}^{m}(\mathbf{z})} = \exp\left(\frac{\|\mathbf{v}\|_2^2 - 2\langle \mathbf{z}, \mathbf{v}\rangle}{2\sigma^2}\right).$$

- Depends on $\mathbf{z}$ only through $\langle \mathbf{z}, \mathbf{v}\rangle$: a one-dimensional problem.
- $\langle \mathbf{z}, \mathbf{v}\rangle$ is like a 1D Gaussian of width $\sigma\|\mathbf{v}\|_2$: $|\langle \mathbf{z}, \mathbf{v}\rangle| \leq 12\sigma\|\mathbf{v}\|_2$ except w.p. 2^{-100}, and then the ratio is at most M.

Why Gaussian Rejection Sampling Works

The ratio between the target and the real distribution is

$$\frac{\mathcal{D}^m_\sigma(\mathbf{z})}{\mathcal{D}^m_{\mathbf{v},\sigma}(\mathbf{z})} = \exp\left(\frac{\|\mathbf{v}\|_2^2 - 2\langle \mathbf{z}, \mathbf{v}\rangle}{2\sigma^2}\right).$$

- Depends on $\mathbf{z}$ only through $\langle \mathbf{z}, \mathbf{v}\rangle$: a one-dimensional problem.
- $\langle \mathbf{z}, \mathbf{v}\rangle$ is like a 1D Gaussian of width $\sigma\|\mathbf{v}\|_2$: $|\langle \mathbf{z}, \mathbf{v}\rangle| \leq 12\sigma\|\mathbf{v}\|_2$ except w.p. 2^{-100}, and then the ratio is at most M.
- Our shift: $\mathbf{v} = \mathbf{Sc}$, $\|\mathbf{Sc}\|_2 \leq \eta\omega\sqrt{m} =: T$. For $\sigma = 12T$: $M = e^{1+1/288} \approx 2.72$.

Why Gaussian Rejection Sampling Works

The ratio between the target and the real distribution is

$$\frac{\mathcal{D}^m_\sigma(\mathbf{z})}{\mathcal{D}^m_{\mathbf{v},\sigma}(\mathbf{z})} = \exp\left(\frac{\|\mathbf{v}\|_2^2 - 2\langle \mathbf{z}, \mathbf{v}\rangle}{2\sigma^2}\right).$$

- Depends on $\mathbf{z}$ only through $\langle \mathbf{z}, \mathbf{v}\rangle$: a one-dimensional problem.
- $\langle \mathbf{z}, \mathbf{v}\rangle$ is like a 1D Gaussian of width $\sigma\|\mathbf{v}\|_2$: $|\langle \mathbf{z}, \mathbf{v}\rangle| \leq 12\sigma\|\mathbf{v}\|_2$ except w.p. 2^{-100}, and then the ratio is at most M.
- Our shift: $\mathbf{v} = \mathbf{Sc}$, $\|\mathbf{Sc}\|_2 \leq \eta\omega\sqrt{m} =: T$. For $\sigma = 12T$: $M = e^{1+1/288} \approx 2.72$.

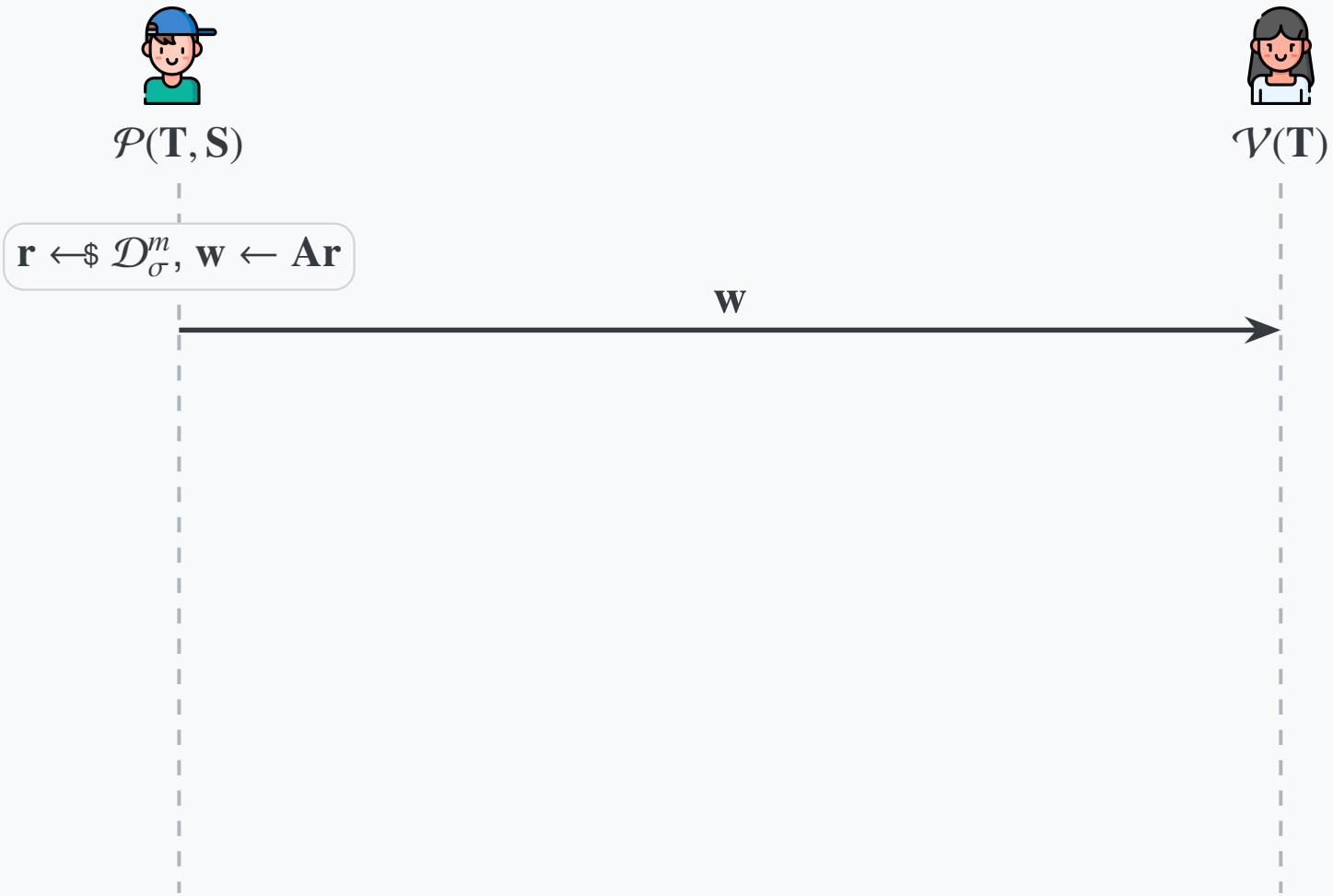
Rejection sampling algorithm

$\mathsf{Rej}(\mathbf{z}, \mathbf{v}, \sigma) \to \{0, 1\}$:

1: $u \leftarrow\$ [0, 1)$;

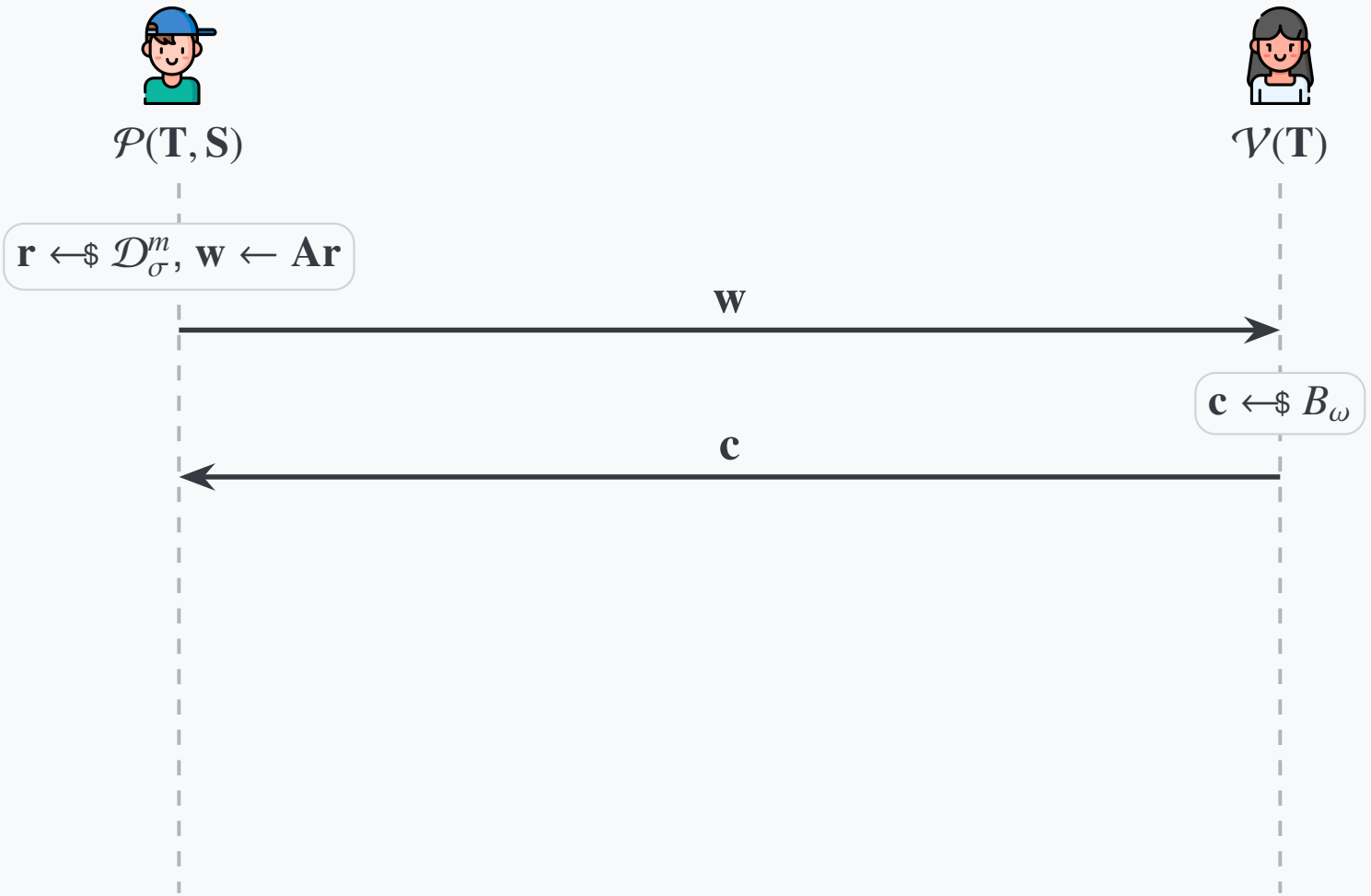
2: **return** $\left[\!\left[u \leq \frac{1}{M}\exp\left(\frac{-2\langle \mathbf{z},\mathbf{v}\rangle + \|\mathbf{v}\|_2^2}{2\sigma^2}\right)\right]\!\right]$. *// 1 means accept*

Lyubashevsky Identification Protocol



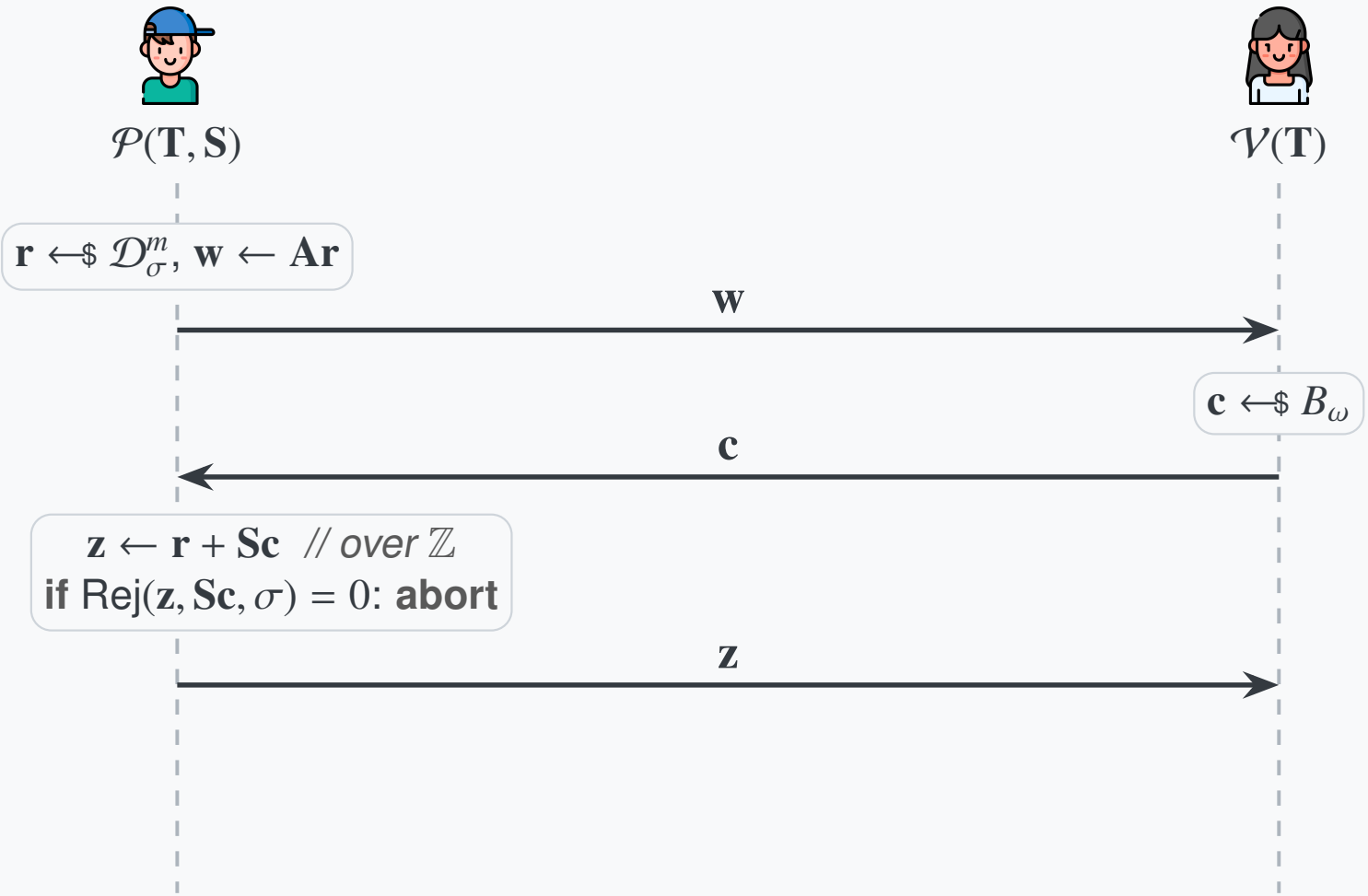
Here, $\mathbf{S} \leftarrow\$ S_\eta^{m \times k}$, $\mathbf{T} = \mathbf{AS}$, and $\sigma = 12\eta\omega\sqrt{m}$. An abort means that both parties restart.

Lyubashevsky Identification Protocol



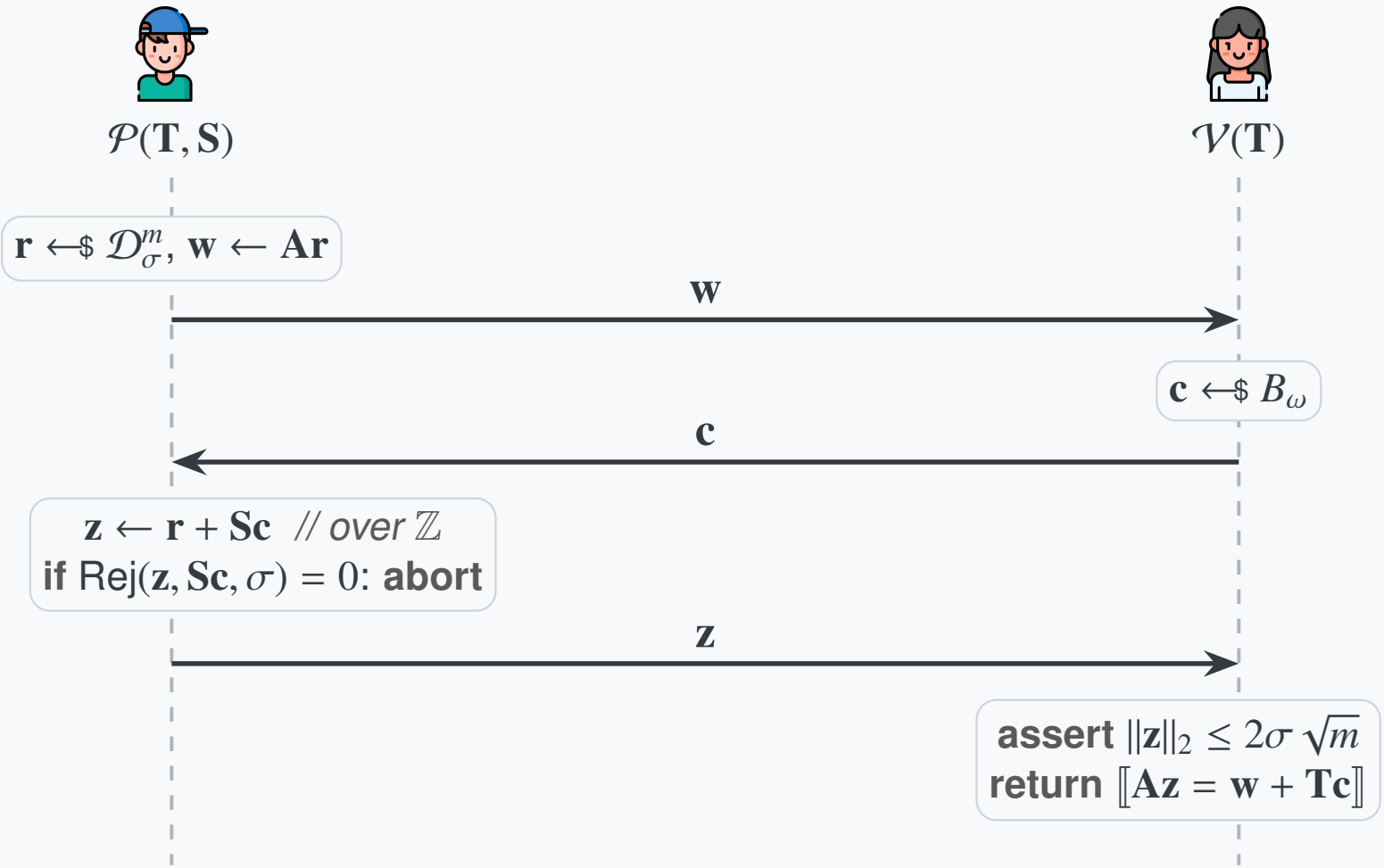
Here, $\mathbf{S} \leftarrow\$ S_\eta^{m \times k}$, $\mathbf{T} = \mathbf{AS}$, and $\sigma = 12\eta\omega\sqrt{m}$. An abort means that both parties restart.

Lyubashevsky Identification Protocol



Here, $\mathbf{S} \leftarrow\$ S_\eta^{m \times k}$, $\mathbf{T} = \mathbf{AS}$, and $\sigma = 12\eta\omega\sqrt{m}$. An abort means that both parties restart.

Lyubashevsky Identification Protocol



Here, $\mathbf{S} \leftarrow\$ S_\eta^{m \times k}$, $\mathbf{T} = \mathbf{AS}$, and $\sigma = 12\eta\omega\sqrt{m}$. An abort means that both parties restart.

Lyubashevsky Protocol: Security

- **Completeness.** $\mathbf{Az} = \mathbf{w} + \mathbf{Tc}$ as before. Accepted $\mathbf{z}$ is distributed as $\mathcal{D}_\sigma^m$, so the norm check fails with negligible probability. $M \approx 2.72$ attempts on average.

Lyubashevsky Protocol: Security

- **Completeness.** $\mathbf{Az} = \mathbf{w} + \mathbf{Tc}$ as before. Accepted $\mathbf{z}$ is distributed as $\mathcal{D}_\sigma^m$, so the norm check fails with negligible probability. $M \approx 2.72$ attempts on average.
- **Zero-knowledge.** Conditioned on no abort, $\mathbf{z} \sim \mathcal{D}_\sigma^m$ independently of $\mathbf{S}$. The simulator samples $\mathbf{c} \leftarrow\!\$\ B_\omega$, $\mathbf{z} \leftarrow\!\$\ \mathcal{D}_\sigma^m$ and sets $\mathbf{w} := \mathbf{Az} - \mathbf{Tc}$.

Lyubashevsky Protocol: Security

- **Completeness.** $\mathbf{Az} = \mathbf{w} + \mathbf{Tc}$ as before. Accepted $\mathbf{z}$ is distributed as $\mathcal{D}_\sigma^m$, so the norm check fails with negligible probability. $M \approx 2.72$ attempts on average.
- **Zero-knowledge.** Conditioned on no abort, $\mathbf{z} \sim \mathcal{D}_\sigma^m$ independently of $\mathbf{S}$. The simulator samples $\mathbf{c} \leftarrow\!\$\ B_\omega$, $\mathbf{z} \leftarrow\!\$\ \mathcal{D}_\sigma^m$ and sets $\mathbf{w} := \mathbf{Az} - \mathbf{Tc}$.
- **Soundness.** The extraction argument of Attempt #2.

Lyubashevsky Protocol: Security

- **Completeness.** $\mathbf{Az} = \mathbf{w} + \mathbf{Tc}$ as before. Accepted $\mathbf{z}$ is distributed as $\mathcal{D}_\sigma^m$, so the norm check fails with negligible probability. $M \approx 2.72$ attempts on average.
- **Zero-knowledge.** Conditioned on no abort, $\mathbf{z} \sim \mathcal{D}_\sigma^m$ independently of $\mathbf{S}$. The simulator samples $\mathbf{c} \leftarrow\!\$\ B_\omega$, $\mathbf{z} \leftarrow\!\$\ \mathcal{D}_\sigma^m$ and sets $\mathbf{w} := \mathbf{Az} - \mathbf{Tc}$.
- **Soundness.** The extraction argument of Attempt #2.

Relaxed extraction

The extractor does not obtain $\mathbf{S}$, but the pair $(\bar{\mathbf{z}}, \bar{\mathbf{c}}) := (\mathbf{z} - \mathbf{z}', \mathbf{c} - \mathbf{c}')$ with $\mathbf{A}\bar{\mathbf{z}} = \mathbf{T}\bar{\mathbf{c}}$:

- $\bar{\mathbf{z}}$ is about σ/η times longer than the honest secret;
- the relation holds only after multiplying $\mathbf{T}$ by $\bar{\mathbf{c}}$.

This gap (*soundness slack*) is inherent to lattice-based proofs of knowledge.

Lyubashevsky Signature Scheme

Fiat-Shamir: $\mathbf{c} \leftarrow \mathsf{H}(\mu, \mathbf{w})$ for $\mathsf{H} : \{0,1\}^* \to B_\omega$; on rejection, start over.

KeyGen(1^λ):

1: $\mathbf{S} \leftarrow\!\!\$\ S_\eta^{m \times k}$;
2: $\mathbf{T} \leftarrow \mathbf{AS}$;
3: **return** $(\mathbf{T}, \mathbf{S})$.

Lyubashevsky Signature Scheme

Fiat-Shamir: $\mathbf{c} \leftarrow \mathsf{H}(\mu, \mathbf{w})$ for $\mathsf{H} : \{0,1\}^* \to B_\omega$; on rejection, start over.

KeyGen(1^λ):

1: $\mathbf{S} \leftarrow\!\$\; S_\eta^{m \times k}$;
2: $\mathbf{T} \leftarrow \mathbf{AS}$;
3: **return** $(\mathbf{T}, \mathbf{S})$.

Sign$(\mathbf{S}, \mu) \to$ sig:

1: $\mathbf{r} \leftarrow\!\$\; \mathcal{D}_\sigma^m$;
2: $\mathbf{c} \leftarrow \mathsf{H}(\mu, \mathbf{Ar} \bmod q)$;
3: $\mathbf{z} \leftarrow \mathbf{r} + \mathbf{Sc}$; *// over* $\mathbb{Z}$
4: **if** $\mathsf{Rej}(\mathbf{z}, \mathbf{Sc}, \sigma) = 0$: **goto** 1;
5: **return** sig $= (\mathbf{z}, \mathbf{c})$.

Lyubashevsky Signature Scheme

Fiat-Shamir: $\mathbf{c} \leftarrow \mathsf{H}(\mu, \mathbf{w})$ for $\mathsf{H} : \{0,1\}^* \to B_\omega$; on rejection, start over.

KeyGen(1^λ):

1: $\mathbf{S} \leftarrow\!\$\; S_\eta^{m \times k}$;
2: $\mathbf{T} \leftarrow \mathbf{AS}$;
3: **return** $(\mathbf{T}, \mathbf{S})$.

Sign$(\mathbf{S}, \mu) \to$ sig:

1: $\mathbf{r} \leftarrow\!\$\; \mathcal{D}_\sigma^m$;
2: $\mathbf{c} \leftarrow \mathsf{H}(\mu, \mathbf{Ar} \bmod q)$;
3: $\mathbf{z} \leftarrow \mathbf{r} + \mathbf{Sc}$; *// over* $\mathbb{Z}$
4: **if** $\mathsf{Rej}(\mathbf{z}, \mathbf{Sc}, \sigma) = 0$: **goto** 1;
5: **return** sig $= (\mathbf{z}, \mathbf{c})$.

Verify$(\mathbf{T}, \mu,$ sig$)$:

1: Parse $(\mathbf{z}, \mathbf{c}) \leftarrow$ sig;
2: $\mathbf{w} \leftarrow \mathbf{Az} - \mathbf{Tc} \bmod q$;
3: **assert** $\|\mathbf{z}\|_2 \le 2\sigma\sqrt{m}$;
4: **return** $[\![\mathbf{c} = \mathsf{H}(\mu, \mathbf{w})]\!]$.

Lyubashevsky Signature Scheme

Fiat-Shamir: $\mathbf{c} \leftarrow \mathsf{H}(\mu, \mathbf{w})$ for $\mathsf{H} : \{0,1\}^* \to B_\omega$; on rejection, start over.

KeyGen(1^λ):

1: $\mathbf{S} \leftarrow\!\$\ S_\eta^{m \times k}$;
2: $\mathbf{T} \leftarrow \mathbf{AS}$;
3: **return** $(\mathbf{T}, \mathbf{S})$.

Sign$(\mathbf{S}, \mu) \to$ sig:

1: $\mathbf{r} \leftarrow\!\$\ \mathcal{D}_\sigma^m$;
2: $\mathbf{c} \leftarrow \mathsf{H}(\mu, \mathbf{Ar} \bmod q)$;
3: $\mathbf{z} \leftarrow \mathbf{r} + \mathbf{Sc}$; *// over* $\mathbb{Z}$
4: **if** $\mathsf{Rej}(\mathbf{z}, \mathbf{Sc}, \sigma) = 0$: **goto** 1;
5: **return** sig $= (\mathbf{z}, \mathbf{c})$.

Verify$(\mathbf{T}, \mu,$ sig$)$:

1: Parse $(\mathbf{z}, \mathbf{c}) \leftarrow$ sig;
2: $\mathbf{w} \leftarrow \mathbf{Az} - \mathbf{Tc} \bmod q$;
3: **assert** $\|\mathbf{z}\|_2 \le 2\sigma\sqrt{m}$;
4: **return** $[\![\mathbf{c} = \mathsf{H}(\mu, \mathbf{w})]\!]$.

Performance

For ≈ 100 bits of security: $n = 512$, $q = 2^{27}$, $m = 8786$, $k = 80$, $\eta = 1$, $\omega = 28$, $\sigma \approx 31495$. Signatures ≈ 19.9 KB, keys ≈ 128 KB each: far from practical!

Lyubashevsky Signature Scheme

Fiat-Shamir: $\mathbf{c} \leftarrow \mathsf{H}(\mu, \mathbf{w})$ for $\mathsf{H} : \{0,1\}^* \to B_\omega$; on rejection, start over.

KeyGen(1^λ):

1: $\mathbf{S} \leftarrow\!\$\ S_\eta^{m \times k}$;
2: $\mathbf{T} \leftarrow \mathbf{AS}$;
3: **return** $(\mathbf{T}, \mathbf{S})$.

Sign$(\mathbf{S}, \mu) \to$ sig:

1: $\mathbf{r} \leftarrow\!\$\ \mathcal{D}_\sigma^m$;
2: $\mathbf{c} \leftarrow \mathsf{H}(\mu, \mathbf{Ar} \bmod q)$;
3: $\mathbf{z} \leftarrow \mathbf{r} + \mathbf{Sc}$; *// over* $\mathbb{Z}$
4: **if** $\mathsf{Rej}(\mathbf{z}, \mathbf{Sc}, \sigma) = 0$: **goto** 1;
5: **return** sig $= (\mathbf{z}, \mathbf{c})$.

Verify$(\mathbf{T}, \mu,$ sig$)$:

1: Parse $(\mathbf{z}, \mathbf{c}) \leftarrow$ sig;
2: $\mathbf{w} \leftarrow \mathbf{Az} - \mathbf{Tc} \bmod q$;
3: **assert** $\|\mathbf{z}\|_2 \le 2\sigma\sqrt{m}$;
4: **return** $[\![\mathbf{c} = \mathsf{H}(\mu, \mathbf{w})]\!]$.

Performance

For ≈ 100 bits of security: $n = 512$, $q = 2^{27}$, $m = 8786$, $k = 80$, $\eta = 1$, $\omega = 28$, $\sigma \approx 31495$. Signatures ≈ 19.9 KB, keys ≈ 128 KB each: far from practical!

Module-SIS/LWE, uniform masks and compression give ML-DSA: ≈ 2.4 KB signatures, ≈ 1.3 KB keys.

Product Proofs

Beyond Knowledge of a Secret Key

ZK applications need more: commit to secret values and prove relations among them.

Example

A confidential transaction proves that each amount is in range, which boils down to *quadratic relations* $b_i(1 - b_i) = 0$ on its bits.

Beyond Knowledge of a Secret Key

ZK applications need more: commit to secret values and prove relations among them.

Example

A confidential transaction proves that each amount is in range, which boils down to *quadratic relations* $b_i(1 - b_i) = 0$ on its bits.

ALS20 (Attema, Lyubashevsky, Seiler) proves multiplicative relations $\mathbf{a} \odot \mathbf{b} = \mathbf{c}$ (for $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{Z}_q^d$) between committed values. Two ingredients:

1. a commitment scheme with an efficient opening proof;
2. linearizing the quadratic constraint using masked openings.

Beyond Knowledge of a Secret Key

ZK applications need more: commit to secret values and prove relations among them.

Example

A confidential transaction proves that each amount is in range, which boils down to *quadratic relations* $b_i(1 - b_i) = 0$ on its bits.

ALS20 (Attema, Lyubashevsky, Seiler) proves multiplicative relations $\mathbf{a} \odot \mathbf{b} = \mathbf{c}$ (for $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{Z}_q^d$) between committed values. Two ingredients:

1. a commitment scheme with an efficient opening proof;
2. linearizing the quadratic constraint using masked openings.

Notation

$\mathcal{R}_q = \mathbb{Z}_q[T]/(T^d + 1)$, d a power of two; bold lowercase letters ($\mathbf{r}$) are vectors over $\mathcal{R}_q$, sans-serif letters (T) are commitments.

The BDLOP Commitment Scheme

Ajtai requires the *message* to be short. BDLOP requires only the *randomness* to be short.

Definition (BDLOP Commitment)

Over the message space $\mathcal{R}_q^n$:

- Setup$(1^\lambda) \to$ pp: sample $\mathbf{B}_0 \leftarrow\$ \mathcal{R}_q^{\mu \times (\lambda+\mu+n)}$ and $\mathbf{b}_1, \ldots, \mathbf{b}_n \leftarrow\$ \mathcal{R}_q^{\lambda+\mu+n}$.
- Com$(\text{pp}, \mathbf{m} \in \mathcal{R}_q^n) \to (\mathsf{T}, \mathbf{r})$: choose $\mathbf{r} \leftarrow\$ \chi^{\lambda+\mu+n}$ and compute

$$\mathbf{t}_0 \leftarrow \mathbf{B}_0\mathbf{r}, \quad t_i \leftarrow \langle \mathbf{b}_i, \mathbf{r} \rangle + m_i \text{ for each } i \in [n].$$

  Output $\mathsf{T} \leftarrow (\mathbf{t}_0, t_1, \ldots, t_n)$ and $\mathbf{r}$.

Here, μ and λ are the ranks of the underlying MSIS and MLWE instances.

BDLOP: Properties

The two parts of the commitment play different roles:

- **Binding**: $\mathbf{t}_0 = \mathbf{B}_0\mathbf{r}$ is an Ajtai commitment to the randomness: two short openings $\mathbf{r} \neq \mathbf{r}'$ give the MSIS solution $\mathbf{r} - \mathbf{r}'$. Once $\mathbf{r}$ is fixed, $t_i = \langle \mathbf{b}_i, \mathbf{r} \rangle + m_i$ determines m_i uniquely.

BDLOP: Properties

The two parts of the commitment play different roles:

- **Binding**: $\mathbf{t}_0 = \mathbf{B}_0\mathbf{r}$ is an Ajtai commitment to the randomness: two short openings $\mathbf{r} \neq \mathbf{r}'$ give the MSIS solution $\mathbf{r} - \mathbf{r}'$. Once $\mathbf{r}$ is fixed, $t_i = \langle \mathbf{b}_i, \mathbf{r} \rangle + m_i$ determines m_i uniquely.
- **Hiding**: by MLWE, $(\mathbf{B}_0\mathbf{r}, \langle \mathbf{b}_1, \mathbf{r} \rangle, \ldots, \langle \mathbf{b}_n, \mathbf{r} \rangle)$ looks uniform: a one-time pad on $\mathbf{m}$.

BDLOP: Properties

The two parts of the commitment play different roles:

- **Binding**: $\mathbf{t}_0 = \mathbf{B}_0\mathbf{r}$ is an Ajtai commitment to the randomness: two short openings $\mathbf{r} \neq \mathbf{r}'$ give the MSIS solution $\mathbf{r} - \mathbf{r}'$. Once $\mathbf{r}$ is fixed, $t_i = \langle \mathbf{b}_i, \mathbf{r} \rangle + m_i$ determines m_i uniquely.
- **Hiding**: by MLWE, $(\mathbf{B}_0\mathbf{r}, \langle \mathbf{b}_1, \mathbf{r} \rangle, \ldots, \langle \mathbf{b}_n, \mathbf{r} \rangle)$ looks uniform: a one-time pad on $\mathbf{m}$.
- Each extra message costs just one $t_i \in \mathcal{R}_q$.

BDLOP: Properties

The two parts of the commitment play different roles:

- **Binding**: $\mathbf{t}_0 = \mathbf{B}_0\mathbf{r}$ is an Ajtai commitment to the randomness: two short openings $\mathbf{r} \neq \mathbf{r}'$ give the MSIS solution $\mathbf{r} - \mathbf{r}'$. Once $\mathbf{r}$ is fixed, $t_i = \langle \mathbf{b}_i, \mathbf{r} \rangle + m_i$ determines m_i uniquely.
- **Hiding**: by MLWE, $(\mathbf{B}_0\mathbf{r}, \langle \mathbf{b}_1, \mathbf{r} \rangle, \ldots, \langle \mathbf{b}_n, \mathbf{r} \rangle)$ looks uniform: a one-time pad on $\mathbf{m}$.
- Each extra message costs just one $t_i \in \mathcal{R}_q$.
- Additively homomorphic: for public $\alpha_1, \alpha_2 \in \mathcal{R}_q$, $\alpha_1 t_1 + \alpha_2 t_2$ commits to $\alpha_1 m_1 + \alpha_2 m_2$ with the vector $\alpha_1 \mathbf{b}_1 + \alpha_2 \mathbf{b}_2$.

The Opening Proof

Prove knowledge of a short $\mathbf{r}$ with $\mathbf{B}_0\mathbf{r} = \mathbf{t}_0$: Lyubashevsky over $\mathcal{R}_q$, with challenges from a set $C \subseteq \mathcal{R}_q$ of polynomials with small coefficients.





$\mathcal{P}(\mathbf{pp}, \mathbb{x} = (\mathbf{T}), \mathbb{w} = (\mathbf{m}, \mathbf{r}))$

$\mathcal{V}(\mathsf{pp}, \mathbb{x} = (\mathsf{T}))$

$\mathbf{y} \leftarrow\$ \mathcal{D}_\sigma^{(\lambda+\mu+n)d}, \mathbf{w} \leftarrow \mathbf{B}_0\mathbf{y}$

$\mathbf{w}$

The Opening Proof

Prove knowledge of a short $\mathbf{r}$ with $\mathbf{B}_0\mathbf{r} = \mathbf{t}_0$: Lyubashevsky over $\mathcal{R}_q$, with challenges from a set $C \subseteq \mathcal{R}_q$ of polynomials with small coefficients.





$\mathcal{P}(\mathbf{pp}, \mathbb{x} = (\mathbf{T}), \mathbb{w} = (\mathbf{m}, \mathbf{r}))$

$\mathcal{V}(\mathsf{pp}, \mathbb{x} = (\mathsf{T}))$

$\mathbf{y} \leftarrow\$ \mathcal{D}_\sigma^{(\lambda+\mu+n)d}, \mathbf{w} \leftarrow \mathbf{B}_0\mathbf{y}$

$\mathbf{w}$

$c \leftarrow\$ C$

c

The Opening Proof

Prove knowledge of a short $\mathbf{r}$ with $\mathbf{B}_0\mathbf{r} = \mathbf{t}_0$: Lyubashevsky over $\mathcal{R}_q$, with challenges from a set $C \subseteq \mathcal{R}_q$ of polynomials with small coefficients.





$\mathcal{P}(\mathbf{pp}, \mathbb{x} = (\mathbf{T}), \mathbb{w} = (\mathbf{m}, \mathbf{r}))$

$\mathcal{V}(\mathrm{pp}, \mathbb{x} = (\mathrm{T}))$

$\mathbf{y} \leftarrow\$ \mathcal{D}_\sigma^{(\lambda+\mu+n)d}$, $\mathbf{w} \leftarrow \mathbf{B}_0\mathbf{y}$

$\mathbf{w}$ →

$c \leftarrow\$ C$

← c

$\mathbf{z} \leftarrow \mathbf{y} + c\mathbf{r}$

if $\mathrm{Rej}(\mathbf{z}, c\mathbf{r}, \sigma) = 0$: **abort**

$\mathbf{z}$ →

The Opening Proof

Prove knowledge of a short $\mathbf{r}$ with $\mathbf{B}_0\mathbf{r} = \mathbf{t}_0$: Lyubashevsky over $\mathcal{R}_q$, with challenges from a set $C \subseteq \mathcal{R}_q$ of polynomials with small coefficients.



$\mathcal{P}(\mathbf{pp}, \mathbb{x} = (\mathbf{T}), \mathbb{w} = (\mathbf{m}, \mathbf{r}))$



$\mathcal{V}(\mathrm{pp}, \mathbb{x} = (\mathrm{T}))$

$\mathbf{y} \leftarrow\$ \mathcal{D}_\sigma^{(\lambda+\mu+n)d}$, $\mathbf{w} \leftarrow \mathbf{B}_0\mathbf{y}$

$\mathbf{w}$ →

$c \leftarrow\$ C$

← c

$\mathbf{z} \leftarrow \mathbf{y} + c\mathbf{r}$
if $\mathrm{Rej}(\mathbf{z}, c\mathbf{r}, \sigma) = 0$: **abort**

$\mathbf{z}$ →

assert $\|\mathbf{z}\|_2 \leq 2\sigma\sqrt{(\lambda+\mu+n)d}$
return $[\![\mathbf{B}_0\mathbf{z} = \mathbf{w} + c\mathbf{t}_0]\!]$

Opening Proof: Relaxed Openings

Two accepting transcripts give $\mathbf{B}_0\bar{\mathbf{z}} = \bar{c}\mathbf{t}_0$ for $\bar{\mathbf{z}} := \mathbf{z} - \mathbf{z}'$ and $\bar{c} := c - c'$.

- If $\bar{c}$ is invertible in $\mathcal{R}_q$: $\mathbf{r}^* := \bar{c}^{-1}\bar{\mathbf{z}}$ and $m_i^* := t_i - \langle \mathbf{b}_i, \mathbf{r}^* \rangle$.

Opening Proof: Relaxed Openings

Two accepting transcripts give $\mathbf{B}_0\bar{\mathbf{z}} = \bar{c}\mathbf{t}_0$ for $\bar{\mathbf{z}} := \mathbf{z} - \mathbf{z}'$ and $\bar{c} := c - c'$.

- If $\bar{c}$ is invertible in $\mathcal{R}_q$: $\mathbf{r}^* := \bar{c}^{-1}\bar{\mathbf{z}}$ and $m_i^* := t_i - \langle \mathbf{b}_i, \mathbf{r}^* \rangle$.
- $\mathbf{r}^*$ is not necessarily short, but $\bar{c}\mathbf{r}^* = \bar{\mathbf{z}}$ is, and this *relaxed opening* is still binding under MSIS.

Opening Proof: Relaxed Openings

Two accepting transcripts give $\mathbf{B}_0\bar{\mathbf{z}} = \bar{c}\mathbf{t}_0$ for $\bar{\mathbf{z}} := \mathbf{z} - \mathbf{z}'$ and $\bar{c} := c - c'$.

- If $\bar{c}$ is invertible in $\mathcal{R}_q$: $\mathbf{r}^* := \bar{c}^{-1}\bar{\mathbf{z}}$ and $m_i^* := t_i - \langle \mathbf{b}_i, \mathbf{r}^* \rangle$.
- $\mathbf{r}^*$ is not necessarily short, but $\bar{c}\mathbf{r}^* = \bar{\mathbf{z}}$ is, and this *relaxed opening* is still binding under MSIS.

Question #3

Let $f := 1 - T \in \mathcal{R}_q$ (note that $\|f\|_\infty = 1$). Compute $\|f^{-1}\|_\infty$. Deduce that the inverse of a short element may be large.

Masked Openings

Crucial observation (ALS20). Once $\mathbf{w}$ is sent, the prover is also committed to the mask $\mathbf{y}^* := \mathbf{z} - c\mathbf{r}^*$: unless it solves MSIS, every accepting response has the form $\mathbf{z} = \mathbf{y}^* + c\mathbf{r}^*$ with the *same* $\mathbf{y}^*$.

Masked Openings

Crucial observation (ALS20). Once $\mathbf{w}$ is sent, the prover is also committed to the mask $\mathbf{y}^* := \mathbf{z} - c\mathbf{r}^*$: unless it solves MSIS, every accepting response has the form $\mathbf{z} = \mathbf{y}^* + c\mathbf{r}^*$ with the *same* $\mathbf{y}^*$.

The verifier can compute from public data

$$f_i := \langle \mathbf{b}_i, \mathbf{z} \rangle - ct_i = a_i - cm_i, \quad \text{where } a_i := \langle \mathbf{b}_i, \mathbf{y} \rangle.$$

Masked Openings

Crucial observation (ALS20). Once $\mathbf{w}$ is sent, the prover is also committed to the mask $\mathbf{y}^* := \mathbf{z} - c\mathbf{r}^*$: unless it solves MSIS, every accepting response has the form $\mathbf{z} = \mathbf{y}^* + c\mathbf{r}^*$ with the *same* $\mathbf{y}^*$.

The verifier can compute from public data

$$f_i := \langle \mathbf{b}_i, \mathbf{z} \rangle - ct_i = a_i - cm_i, \quad \text{where } a_i := \langle \mathbf{b}_i, \mathbf{y} \rangle.$$

- f_i is a **masked opening** of m_i: an affine function of c whose leading coefficient is the message, masked by a_i fixed before seeing c.
- Previous product proofs had to commit to separate masks a_i; here they come for free.

Masked Openings

Crucial observation (ALS20). Once $\mathbf{w}$ is sent, the prover is also committed to the mask $\mathbf{y}^* := \mathbf{z} - c\mathbf{r}^*$: unless it solves MSIS, every accepting response has the form $\mathbf{z} = \mathbf{y}^* + c\mathbf{r}^*$ with the *same* $\mathbf{y}^*$.

The verifier can compute from public data

$$f_i := \langle \mathbf{b}_i, \mathbf{z} \rangle - ct_i = a_i - cm_i, \quad \text{where } a_i := \langle \mathbf{b}_i, \mathbf{y} \rangle.$$

- f_i is a **masked opening** of m_i: an affine function of c whose leading coefficient is the message, masked by a_i fixed before seeing c.
- Previous product proofs had to commit to separate masks a_i; here they come for free.

Question #4

Verify that indeed $\langle \mathbf{b}_i, \mathbf{z} \rangle - ct_i = \langle \mathbf{b}_i, \mathbf{y} \rangle - cm_i$ for every $i \in [n]$.

Product Proof: the Idea

Goal: prove $m_1 m_2 = m_3$ for committed $m_1, m_2, m_3 \in \mathcal{R}_q$. Combine the masked openings:

$$\begin{aligned} f_1 f_2 + c f_3 &= (a_1 - c m_1)(a_2 - c m_2) + c(a_3 - c m_3) \\ &= (m_1 m_2 - m_3)c^2 + (a_3 - a_1 m_2 - a_2 m_1)c + a_1 a_2. \end{aligned}$$

Product Proof: the Idea

Goal: prove $m_1 m_2 = m_3$ for committed $m_1, m_2, m_3 \in \mathcal{R}_q$. Combine the masked openings:

$$\begin{aligned} f_1 f_2 + c f_3 &= (a_1 - c m_1)(a_2 - c m_2) + c(a_3 - c m_3) \\ &= (m_1 m_2 - m_3) c^2 + (a_3 - a_1 m_2 - a_2 m_1) c + a_1 a_2. \end{aligned}$$

If the relation holds, the expression is **linear** in c. How to check linearity in zero-knowledge?

Product Proof: the Idea

Goal: prove $m_1 m_2 = m_3$ for committed $m_1, m_2, m_3 \in \mathcal{R}_q$. Combine the masked openings:

$$\begin{aligned} f_1 f_2 + c f_3 &= (a_1 - c m_1)(a_2 - c m_2) + c(a_3 - c m_3) \\ &= (m_1 m_2 - m_3)c^2 + (a_3 - a_1 m_2 - a_2 m_1)c + a_1 a_2. \end{aligned}$$

If the relation holds, the expression is **linear** in c. How to check linearity in zero-knowledge?

Garbage commitment

Before seeing c, the prover commits to $g := a_3 - a_1 m_2 - a_2 m_1$ as a fourth message, $t_4 := \langle \mathbf{b}_4, \mathbf{r} \rangle + g$, and sends $v := a_1 a_2 + a_4$ for $a_4 := \langle \mathbf{b}_4, \mathbf{y} \rangle$.

Product Proof: the Idea

Goal: prove $m_1 m_2 = m_3$ for committed $m_1, m_2, m_3 \in \mathcal{R}_q$. Combine the masked openings:

$$\begin{aligned} f_1 f_2 + c f_3 &= (a_1 - c m_1)(a_2 - c m_2) + c(a_3 - c m_3) \\ &= (m_1 m_2 - m_3)c^2 + (a_3 - a_1 m_2 - a_2 m_1)c + a_1 a_2. \end{aligned}$$

If the relation holds, the expression is **linear** in c. How to check linearity in zero-knowledge?

Garbage commitment

Before seeing c, the prover commits to $g := a_3 - a_1 m_2 - a_2 m_1$ as a fourth message, $t_4 := \langle \mathbf{b}_4, \mathbf{r} \rangle + g$, and sends $v := a_1 a_2 + a_4$ for $a_4 := \langle \mathbf{b}_4, \mathbf{y} \rangle$.

Since $f_4 = a_4 - cg$,

$$f_1 f_2 + c f_3 + f_4 = (m_1 m_2 - m_3)c^2 + v.$$

The verifier accepts iff $f_1 f_2 + c f_3 + f_4 = v$.

The ALS20 Product Proof

$\mathcal{P}(\mathbf{pp}, \mathbb{x} = (\mathbf{T}), \mathbb{w} = (m_1, m_2, m_3, \mathbf{r}))$:

1: $\mathbf{y} \leftarrow\!\$\ \mathcal{D}_\sigma^{(\lambda+\mu+4)d}$;
2: $a_i \leftarrow \langle \mathbf{b}_i, \mathbf{y} \rangle$ for $i = 1, \ldots, 4$;
3: *// garbage term*
$t_4 \leftarrow \langle \mathbf{b}_4, \mathbf{r} \rangle + a_3 - a_1 m_2 - a_2 m_1$;
4: $\mathbf{w} \leftarrow \mathbf{B}_0 \mathbf{y}$ and $v \leftarrow a_1 a_2 + a_4$;
5: **send** $(t_4, \mathbf{w}, v)$;

$\mathcal{V}(\mathsf{pp}, \mathbb{x} = (\mathsf{T})) \rightarrow \{0, 1\}$:

1: **receive** $(t_4, \mathbf{w}, v)$;
2: **send** $c \leftarrow\!\$\ C$;

The ALS20 Product Proof

$\mathcal{P}(\mathbf{pp}, \mathbb{x} = (\mathbf{T}), \mathbb{w} = (m_1, m_2, m_3, \mathbf{r}))$:

1: $\mathbf{y} \leftarrow\!\$\ \mathcal{D}_\sigma^{(\lambda+\mu+4)d}$;
2: $a_i \leftarrow \langle \mathbf{b}_i, \mathbf{y} \rangle$ for $i = 1, \ldots, 4$;
3: *// garbage term*
$t_4 \leftarrow \langle \mathbf{b}_4, \mathbf{r} \rangle + a_3 - a_1 m_2 - a_2 m_1$;
4: $\mathbf{w} \leftarrow \mathbf{B}_0 \mathbf{y}$ and $v \leftarrow a_1 a_2 + a_4$;
5: **send** $(t_4, \mathbf{w}, v)$;
6: **receive** c;
7: $\mathbf{z} \leftarrow \mathbf{y} + c\mathbf{r}$;
8: **if** $\mathrm{Rej}(\mathbf{z}, c\mathbf{r}, \sigma) = 0$: **abort**;
9: **send** $\mathbf{z}$.

$\mathcal{V}(\mathrm{pp}, \mathbb{x} = (\mathsf{T})) \to \{0, 1\}$:

1: **receive** $(t_4, \mathbf{w}, v)$;
2: **send** $c \leftarrow\!\$\ C$;
3: **receive** $\mathbf{z}$;
4: **assert** $\|\mathbf{z}\|_2 \le \beta$;
5: **assert** $\mathbf{B}_0 \mathbf{z} = \mathbf{w} + c\mathbf{t}_0$;
6: $f_i \leftarrow \langle \mathbf{b}_i, \mathbf{z} \rangle - c t_i$ for each i;
7: **return** $[\![f_1 f_2 + c f_3 + f_4 = v]\!]$.

The ALS20 Product Proof

$\mathcal{P}(\mathbf{pp}, \mathbb{x} = (\mathbf{T}), \mathbb{w} = (m_1, m_2, m_3, \mathbf{r}))$:

1: $\mathbf{y} \leftarrow\$ \mathcal{D}_\sigma^{(\lambda+\mu+4)d}$;
2: $a_i \leftarrow \langle \mathbf{b}_i, \mathbf{y} \rangle$ for $i = 1, \ldots, 4$;
3: *// garbage term*
$t_4 \leftarrow \langle \mathbf{b}_4, \mathbf{r} \rangle + a_3 - a_1 m_2 - a_2 m_1$;
4: $\mathbf{w} \leftarrow \mathbf{B}_0 \mathbf{y}$ and $v \leftarrow a_1 a_2 + a_4$;
5: **send** $(t_4, \mathbf{w}, v)$;
6: **receive** c;
7: $\mathbf{z} \leftarrow \mathbf{y} + c\mathbf{r}$;
8: **if** $\mathrm{Rej}(\mathbf{z}, c\mathbf{r}, \sigma) = 0$: **abort**;
9: **send** $\mathbf{z}$.

$\mathcal{V}(\mathrm{pp}, \mathbb{x} = (\mathsf{T})) \to \{0, 1\}$:

1: **receive** $(t_4, \mathbf{w}, v)$;
2: **send** $c \leftarrow\$ C$;
3: **receive** $\mathbf{z}$;
4: **assert** $\|\mathbf{z}\|_2 \leq \beta$;
5: **assert** $\mathbf{B}_0 \mathbf{z} = \mathbf{w} + c\mathbf{t}_0$;
6: $f_i \leftarrow \langle \mathbf{b}_i, \mathbf{z} \rangle - c t_i$ for each i;
7: **return** $[\![f_1 f_2 + c f_3 + f_4 = v]\!]$.

Compared to the opening proof: one additional commitment t_4 and one additional element $v \in \mathcal{R}_q$.

The ALS20 Product Proof

$\mathcal{P}(\mathbf{pp}, \mathbb{x} = (\mathbf{T}), \mathbb{w} = (m_1, m_2, m_3, \mathbf{r}))$:

1: $\mathbf{y} \leftarrow\$ \mathcal{D}_\sigma^{(\lambda+\mu+4)d}$;
2: $a_i \leftarrow \langle \mathbf{b}_i, \mathbf{y} \rangle$ for $i = 1, \ldots, 4$;
3: *// garbage term*
$t_4 \leftarrow \langle \mathbf{b}_4, \mathbf{r} \rangle + a_3 - a_1 m_2 - a_2 m_1$;
4: $\mathbf{w} \leftarrow \mathbf{B}_0 \mathbf{y}$ and $v \leftarrow a_1 a_2 + a_4$;
5: **send** $(t_4, \mathbf{w}, v)$;
6: **receive** c;
7: $\mathbf{z} \leftarrow \mathbf{y} + c\mathbf{r}$;
8: **if** $\mathrm{Rej}(\mathbf{z}, c\mathbf{r}, \sigma) = 0$: **abort**;
9: **send** $\mathbf{z}$.

$\mathcal{V}(\mathrm{pp}, \mathbb{x} = (\mathsf{T})) \to \{0, 1\}$:

1: **receive** $(t_4, \mathbf{w}, v)$;
2: **send** $c \leftarrow\$ C$;
3: **receive** $\mathbf{z}$;
4: **assert** $\|\mathbf{z}\|_2 \leq \beta$;
5: **assert** $\mathbf{B}_0 \mathbf{z} = \mathbf{w} + c\mathbf{t}_0$;
6: $f_i \leftarrow \langle \mathbf{b}_i, \mathbf{z} \rangle - c t_i$ for each i;
7: **return** $[\![f_1 f_2 + c f_3 + f_4 = v]\!]$.

Compared to the opening proof: one additional commitment t_4 and one additional element $v \in \mathcal{R}_q$.

Question #5

Simplify the protocol for proving $m^2 = m$ for a single committed $m \in \mathcal{R}_q$.

Packing $\mathbb{Z}_q$ Values into NTT Slots

Applications need products of *numbers* in $\mathbb{Z}_q$. If $q \equiv 1 \pmod{2d}$, then

$$T^d + 1 = \prod_{j=1}^{d} (T - \zeta_j) \pmod{q}, \quad \text{and} \quad \mathsf{NTT} : m \mapsto (m(\zeta_1), \ldots, m(\zeta_d))$$

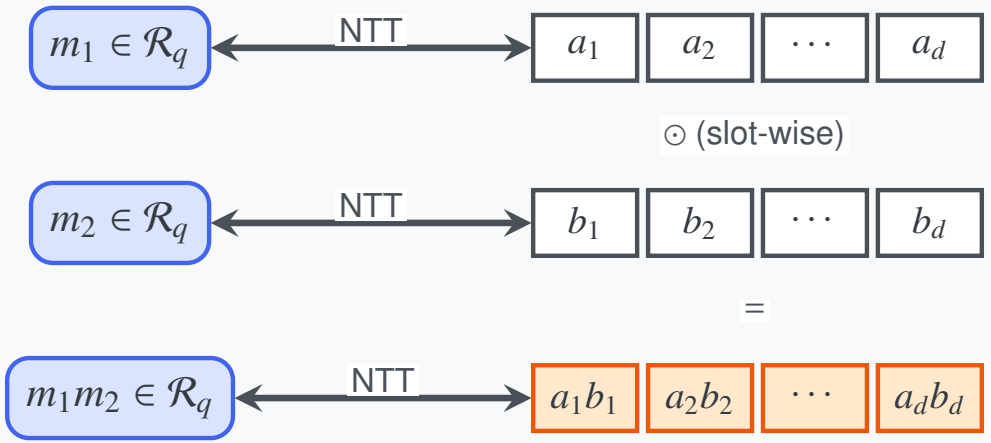
is a ring isomorphism $\mathcal{R}_q \to \mathbb{Z}_q^d$ (CRT), with $\mathsf{NTT}(m_1 m_2) = \mathsf{NTT}(m_1) \odot \mathsf{NTT}(m_2)$.

Packing $\mathbb{Z}_q$ Values into NTT Slots

Applications need products of *numbers* in $\mathbb{Z}_q$. If $q \equiv 1 \pmod{2d}$, then

$$T^d + 1 = \prod_{j=1}^{d} (T - \zeta_j) \pmod{q}, \quad \text{and} \quad \mathrm{NTT} : m \mapsto (m(\zeta_1), \ldots, m(\zeta_d))$$

is a ring isomorphism $\mathcal{R}_q \to \mathbb{Z}_q^d$ (CRT), with $\mathrm{NTT}(m_1 m_2) = \mathrm{NTT}(m_1) \odot \mathrm{NTT}(m_2)$.

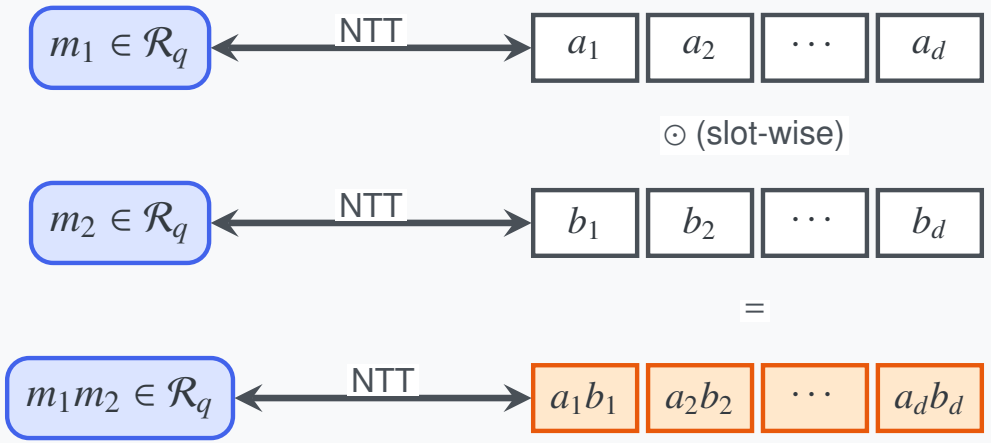


Packing $\mathbb{Z}_q$ Values into NTT Slots

Applications need products of *numbers* in $\mathbb{Z}_q$. If $q \equiv 1 \pmod{2d}$, then

$$T^d + 1 = \prod_{j=1}^{d} (T - \zeta_j) \pmod{q}, \quad \text{and} \quad \mathrm{NTT} : m \mapsto (m(\zeta_1), \ldots, m(\zeta_d))$$

is a ring isomorphism $\mathcal{R}_q \to \mathbb{Z}_q^d$ (CRT), with $\mathrm{NTT}(m_1 m_2) = \mathrm{NTT}(m_1) \odot \mathrm{NTT}(m_2)$.



To prove $a_j b_j = c_j$ for all j: commit to $\mathrm{NTT}^{-1}(\mathbf{a})$, $\mathrm{NTT}^{-1}(\mathbf{b})$, $\mathrm{NTT}^{-1}(\mathbf{c})$ and run the product proof once.

Example: a Range Proof

Example

To prove that $x \in \mathbb{Z}$ lies in range from 0 to $2^d - 1$:

1: find the bit-decomposition $x = \sum_{i \in [d]} x_i 2^i$;
2: form a bit-vector $\mathbf{x} = (x_0, \ldots, x_{d-1})$;
3: compute $f := \mathsf{NTT}^{-1}(\mathbf{x})$; *// interpolation:* $f(\zeta_i) = x_i$
4: use the ALS20 protocol to show that $f^2 = f$.

Example: a Range Proof

Example

To prove that $x \in \mathbb{Z}$ lies in range from 0 to $2^d - 1$:

1: find the bit-decomposition $x = \sum_{i \in [d]} x_i 2^i$;
2: form a bit-vector $\mathbf{x} = (x_0, \ldots, x_{d-1})$;
3: compute $f := \mathsf{NTT}^{-1}(\mathbf{x})$; *// interpolation:* $f(\zeta_i) = x_i$
4: use the ALS20 protocol to show that $f^2 = f$.

Question #6

Let $d = 4$, $q = 17$, so the slots are $(\zeta, \zeta^3, \zeta^5, \zeta^7) = (2, 8, 15, 9)$ for $\zeta = 2$.

1: Verify that $\mathsf{NTT}(9 + 2T^2) = (0, 1, 0, 1)$, encoding $x = 10$.
2: Verify directly that $f^2 = f$ in $\mathcal{R}_{17}$ for $f := 9 + 2T^2$.
3: Which amount is encoded by $1 - f$?

Soundness in Fully Splitting Rings

- Soundness needs invertible challenge differences. In a fully splitting ring, $\bar{c}$ is invertible iff *all* slots $\bar{c}(\zeta_j)$ are non-zero.

Soundness in Fully Splitting Rings

- Soundness needs invertible challenge differences. In a fully splitting ring, $\bar{c}$ is invertible iff *all* slots $\bar{c}(\zeta_j)$ are non-zero.
- A cheating prover guesses a single slot of c with probability $\approx 1/q$ and cheats in that slot (e.g., a non-bit). For $q \approx 2^{32}$, $1/q$ is far from negligible!

Soundness in Fully Splitting Rings

- Soundness needs invertible challenge differences. In a fully splitting ring, $\bar{c}$ is invertible iff *all* slots $\bar{c}(\zeta_j)$ are non-zero.
- A cheating prover guesses a single slot of c with probability $\approx 1/q$ and cheats in that slot (e.g., a non-bit). For $q \approx 2^{32}$, $1/q$ is far from negligible!
- Repeating k times gives q^{-k}, but multiplies masks and garbage terms.

Soundness in Fully Splitting Rings

- Soundness needs invertible challenge differences. In a fully splitting ring, $\bar{c}$ is invertible iff *all* slots $\bar{c}(\zeta_j)$ are non-zero.
- A cheating prover guesses a single slot of c with probability $\approx 1/q$ and cheats in that slot (e.g., a non-bit). For $q \approx 2^{32}$, $1/q$ is far from negligible!
- Repeating k times gives q^{-k}, but multiplies masks and garbage terms.

Automorphism trick (ALS20)

For odd i, $\varphi_i : T \mapsto T^i$ is an automorphism of $\mathcal{R}_q$ permuting the slots. Run k opening proofs with challenges $c, \varphi(c), \ldots, \varphi^{k-1}(c)$ for a single c: one guess no longer suffices. Combine the k relations with random $\alpha_i \in \mathcal{R}_q$: only *one* garbage commitment. Soundness error $\approx (3/q)^k$.

Relations over $\mathbb{Z}_q$

Limitations of ALS20

(i) Every committed message costs one element of $\mathcal{R}_q$: committing to e.g. 2048 ternary coefficients is expensive.

Limitations of ALS20

(i) Every committed message costs one element of $\mathcal{R}_q$: committing to e.g. 2048 ternary coefficients is expensive.

(ii) NTT packing requires a fully splitting ring, hence the automorphism trick.

Limitations of ALS20

(i) Every committed message costs one element of $\mathcal{R}_q$: committing to e.g. 2048 ternary coefficients is expensive.

(ii) NTT packing requires a fully splitting ring, hence the automorphism trick.

(iii) Many statements are not products: e.g., knowledge of $\mathbf{s}$ with $\mathbf{As} = \mathbf{t}$ and $\|\mathbf{s}\|_2 \leq \beta$ *exactly*, not the relaxed statement.

Limitations of ALS20

(i) Every committed message costs one element of $\mathcal{R}_q$: committing to e.g. 2048 ternary coefficients is expensive.

(ii) NTT packing requires a fully splitting ring, hence the automorphism trick.

(iii) Many statements are not products: e.g., knowledge of $\mathbf{s}$ with $\mathbf{As} = \mathbf{t}$ and $\|\mathbf{s}\|_2 \leq \beta$ *exactly*, not the relaxed statement.

LNP22 (Lyubashevsky, Nguyen, Plançon) address all three, with four ideas:

1: a combined commitment scheme (ABDLOP);

Limitations of ALS20

(i) Every committed message costs one element of $\mathcal{R}_q$: committing to e.g. 2048 ternary coefficients is expensive.

(ii) NTT packing requires a fully splitting ring, hence the automorphism trick.

(iii) Many statements are not products: e.g., knowledge of $\mathbf{s}$ with $\mathbf{As} = \mathbf{t}$ and $\|\mathbf{s}\|_2 \leq \beta$ *exactly*, not the relaxed statement.

LNP22 (Lyubashevsky, Nguyen, Plançon) address all three, with four ideas:

1: a combined commitment scheme (ABDLOP);

2: encoding $\mathbb{Z}_q$-relations into *constant coefficients*;

Limitations of ALS20

(i) Every committed message costs one element of $\mathcal{R}_q$: committing to e.g. 2048 ternary coefficients is expensive.

(ii) NTT packing requires a fully splitting ring, hence the automorphism trick.

(iii) Many statements are not products: e.g., knowledge of $\mathbf{s}$ with $\mathbf{As} = \mathbf{t}$ and $\|\mathbf{s}\|_2 \leq \beta$ *exactly*, not the relaxed statement.

LNP22 (Lyubashevsky, Nguyen, Plançon) address all three, with four ideas:

1: a combined commitment scheme (ABDLOP);
2: encoding $\mathbb{Z}_q$-relations into *constant coefficients*;
3: a quadratic proof in the style of ALS20;

Limitations of ALS20

(i) Every committed message costs one element of $\mathcal{R}_q$: committing to e.g. 2048 ternary coefficients is expensive.

(ii) NTT packing requires a fully splitting ring, hence the automorphism trick.

(iii) Many statements are not products: e.g., knowledge of $\mathbf{s}$ with $\mathbf{As} = \mathbf{t}$ and $\|\mathbf{s}\|_2 \leq \beta$ *exactly*, not the relaxed statement.

LNP22 (Lyubashevsky, Nguyen, Plançon) address all three, with four ideas:

1: a combined commitment scheme (ABDLOP);
2: encoding $\mathbb{Z}_q$-relations into *constant coefficients*;
3: a quadratic proof in the style of ALS20;
4: an approximate range proof lifting relations from $\mathbb{Z}_q$ to $\mathbb{Z}$.

The ABDLOP Commitment Scheme

Ajtai: arbitrarily long, but only short messages. BDLOP: arbitrary coefficients, but one ring element each. ABDLOP merges them under a single randomness $\boldsymbol{\rho}$.

Definition (ABDLOP Commitment)

Over $S_\alpha^m \times \mathcal{R}_q^n$ (short messages $\mathbf{x}$, long ones $\mathbf{m}$):

- Setup$(1^\lambda) \to$ pp: $\mathbf{A} \leftarrow\!\$\ \mathcal{R}_q^{\kappa \times m}$, $\mathbf{A}_\rho \leftarrow\!\$\ \mathcal{R}_q^{\kappa \times r}$, $\mathbf{B} \leftarrow\!\$\ \mathcal{R}_q^{n \times r}$.
- Com$(\text{pp}, \mathbf{x}, \mathbf{m}) \to (\mathsf{T}, \boldsymbol{\rho})$: choose $\boldsymbol{\rho} \leftarrow\!\$\ \chi^r$ and compute

$$\mathbf{t}_A \leftarrow \mathbf{A}\mathbf{x} + \mathbf{A}_\rho \boldsymbol{\rho}, \quad \mathbf{t}_B \leftarrow \mathbf{B}\boldsymbol{\rho} + \mathbf{m}.$$

Output $\mathsf{T} \leftarrow (\mathbf{t}_A, \mathbf{t}_B)$ and $\boldsymbol{\rho}$.

The ABDLOP Commitment Scheme

Ajtai: arbitrarily long, but only short messages. BDLOP: arbitrary coefficients, but one ring element each. ABDLOP merges them under a single randomness $\boldsymbol{\rho}$.

Definition (ABDLOP Commitment)

Over $S_\alpha^m \times \mathcal{R}_q^n$ (short messages $\mathbf{x}$, long ones $\mathbf{m}$):

- Setup$(1^\lambda) \to$ pp: $\mathbf{A} \leftarrow\!\$\ \mathcal{R}_q^{\kappa \times m}$, $\mathbf{A}_\rho \leftarrow\!\$\ \mathcal{R}_q^{\kappa \times r}$, $\mathbf{B} \leftarrow\!\$\ \mathcal{R}_q^{n \times r}$.
- Com$(\text{pp}, \mathbf{x}, \mathbf{m}) \to (\mathsf{T}, \boldsymbol{\rho})$: choose $\boldsymbol{\rho} \leftarrow\!\$\ \chi^r$ and compute

$$\mathbf{t}_A \leftarrow \mathbf{A}\mathbf{x} + \mathbf{A}_\rho \boldsymbol{\rho}, \quad \mathbf{t}_B \leftarrow \mathbf{B}\boldsymbol{\rho} + \mathbf{m}.$$

  Output $\mathsf{T} \leftarrow (\mathbf{t}_A, \mathbf{t}_B)$ and $\boldsymbol{\rho}$.

- The size does not depend on the length m of $\mathbf{x}$: put the (long) short witness into $\mathbf{x}$, a few auxiliary values into $\mathbf{m}$.

The ABDLOP Commitment Scheme

Ajtai: arbitrarily long, but only short messages. BDLOP: arbitrary coefficients, but one ring element each. ABDLOP merges them under a single randomness $\boldsymbol{\rho}$.

Definition (ABDLOP Commitment)

Over $S_\alpha^m \times \mathcal{R}_q^n$ (short messages $\mathbf{x}$, long ones $\mathbf{m}$):

- Setup$(1^\lambda) \to$ pp: $\mathbf{A} \leftarrow\$ \mathcal{R}_q^{\kappa \times m}$, $\mathbf{A}_\rho \leftarrow\$ \mathcal{R}_q^{\kappa \times r}$, $\mathbf{B} \leftarrow\$ \mathcal{R}_q^{n \times r}$.
- Com$(\text{pp}, \mathbf{x}, \mathbf{m}) \to (\mathsf{T}, \boldsymbol{\rho})$: choose $\boldsymbol{\rho} \leftarrow\$ \chi^r$ and compute

$$\mathbf{t}_A \leftarrow \mathbf{A}\mathbf{x} + \mathbf{A}_\rho \boldsymbol{\rho}, \quad \mathbf{t}_B \leftarrow \mathbf{B}\boldsymbol{\rho} + \mathbf{m}.$$

Output $\mathsf{T} \leftarrow (\mathbf{t}_A, \mathbf{t}_B)$ and $\boldsymbol{\rho}$.

- The size does not depend on the length m of $\mathbf{x}$: put the (long) short witness into $\mathbf{x}$, a few auxiliary values into $\mathbf{m}$.
- New BDLOP slots can be appended later: $\mathbf{t}_g := \mathbf{B}_g \boldsymbol{\rho} + \mathbf{g}$ under the same randomness.

Opening Proof with Linear Constraints

Prove the opening of T and N linear relations $\mathbf{Lx} + \mathbf{Rm} = \mathbf{s}$ ($\mathbf{L} \in \mathcal{R}_q^{N \times m}$, $\mathbf{R} \in \mathcal{R}_q^{N \times n}$). Masked opening of $\mathbf{m}$: $c\mathbf{t}_B - \mathbf{Bz}_\rho = c\mathbf{m} - \mathbf{By}_\rho$.

$\mathcal{P}(\mathsf{pp}, \mathbb{x} = \mathsf{T}, \mathbb{w} = (\mathbf{x}, \mathbf{m}, \boldsymbol{\rho}))$:

1: $\mathbf{y} \leftarrow\$ \mathcal{D}_\sigma^{md}$, $\mathbf{y}_\rho \leftarrow\$ \mathcal{D}_{\sigma_\rho}^{rd}$;
2: *// commitment mask*
$\mathbf{w} \leftarrow \mathbf{Ay} + \mathbf{A}_\rho \mathbf{y}_\rho$;
3: *// linear-system mask*
$\mathbf{v} \leftarrow \mathbf{Ly} - \mathbf{RBy}_\rho$;
4: **send** $(\mathbf{w}, \mathbf{v})$;

$\mathcal{V}(\mathsf{pp}, \mathbb{x} = (\mathsf{T}, \mathbf{L}, \mathbf{R}, \mathbf{s}))$:

1: **receive** $(\mathbf{w}, \mathbf{v})$;
2: **send** $c \leftarrow\$ C$;

Opening Proof with Linear Constraints

Prove the opening of T and N linear relations $\mathbf{Lx} + \mathbf{Rm} = \mathbf{s}$ ($\mathbf{L} \in \mathcal{R}_q^{N \times m}$, $\mathbf{R} \in \mathcal{R}_q^{N \times n}$). Masked opening of $\mathbf{m}$: $c\mathbf{t}_B - \mathbf{Bz}_\rho = c\mathbf{m} - \mathbf{By}_\rho$.

$\mathcal{P}(\mathsf{pp}, \mathbb{x} = \mathsf{T}, \mathbb{w} = (\mathbf{x}, \mathbf{m}, \boldsymbol{\rho}))$:

1: $\mathbf{y} \leftarrow\$ \mathcal{D}_\sigma^{md}$, $\mathbf{y}_\rho \leftarrow\$ \mathcal{D}_{\sigma_\rho}^{rd}$;
2: *// commitment mask*
$\mathbf{w} \leftarrow \mathbf{Ay} + \mathbf{A}_\rho \mathbf{y}_\rho$;
3: *// linear-system mask*
$\mathbf{v} \leftarrow \mathbf{Ly} - \mathbf{RBy}_\rho$;
4: **send** $(\mathbf{w}, \mathbf{v})$;
5: **receive** c;
6: $\mathbf{z} \leftarrow c\mathbf{x} + \mathbf{y}$, $\mathbf{z}_\rho \leftarrow c\boldsymbol{\rho} + \mathbf{y}_\rho$;
7: **if** $\mathsf{Rej}((\mathbf{z}, \mathbf{z}_\rho), (c\mathbf{x}, c\boldsymbol{\rho}), (\sigma, \sigma_\rho)) = 0$:
abort;
8: **send** $(\mathbf{z}, \mathbf{z}_\rho)$.

$\mathcal{V}(\mathsf{pp}, \mathbb{x} = (\mathsf{T}, \mathbf{L}, \mathbf{R}, \mathbf{s}))$:

1: **receive** $(\mathbf{w}, \mathbf{v})$;
2: **send** $c \leftarrow\$ C$;
3: **receive** $(\mathbf{z}, \mathbf{z}_\rho)$;
4: **assert** $\|\mathbf{z}\|_2 \leq \sigma\sqrt{2md}$;
5: **assert** $\|\mathbf{z}_\rho\|_2 \leq \sigma_\rho\sqrt{2rd}$;
6: **assert** $\mathbf{Az} + \mathbf{A}_\rho\mathbf{z}_\rho - c\mathbf{t}_A = \mathbf{w}$;
7: **return** $[\![\mathbf{Lz} + \mathbf{R}(c\mathbf{t}_B - \mathbf{Bz}_\rho) - c\mathbf{s} = \mathbf{v}]\!]$.

Opening Proof with Linear Constraints

Prove the opening of T and N linear relations $\mathbf{Lx} + \mathbf{Rm} = \mathbf{s}$ ($\mathbf{L} \in \mathcal{R}_q^{N \times m}$, $\mathbf{R} \in \mathcal{R}_q^{N \times n}$). Masked opening of $\mathbf{m}$: $c\mathbf{t}_B - \mathbf{Bz}_\rho = c\mathbf{m} - \mathbf{By}_\rho$.

$\mathcal{P}(\mathsf{pp}, \mathbb{x} = \mathsf{T}, \mathbb{w} = (\mathbf{x}, \mathbf{m}, \boldsymbol{\rho}))$:

1: $\mathbf{y} \leftarrow\$ \mathcal{D}_\sigma^{md}$, $\mathbf{y}_\rho \leftarrow\$ \mathcal{D}_{\sigma_\rho}^{rd}$;
2: *// commitment mask*
$\mathbf{w} \leftarrow \mathbf{Ay} + \mathbf{A}_\rho \mathbf{y}_\rho$;
3: *// linear-system mask*
$\mathbf{v} \leftarrow \mathbf{Ly} - \mathbf{RBy}_\rho$;
4: **send** $(\mathbf{w}, \mathbf{v})$;
5: **receive** c;
6: $\mathbf{z} \leftarrow c\mathbf{x} + \mathbf{y}$, $\mathbf{z}_\rho \leftarrow c\boldsymbol{\rho} + \mathbf{y}_\rho$;
7: **if** $\mathsf{Rej}((\mathbf{z}, \mathbf{z}_\rho), (c\mathbf{x}, c\boldsymbol{\rho}), (\sigma, \sigma_\rho)) = 0$:
abort;
8: **send** $(\mathbf{z}, \mathbf{z}_\rho)$.

$\mathcal{V}(\mathsf{pp}, \mathbb{x} = (\mathsf{T}, \mathbf{L}, \mathbf{R}, \mathbf{s}))$:

1: **receive** $(\mathbf{w}, \mathbf{v})$;
2: **send** $c \leftarrow\$ C$;
3: **receive** $(\mathbf{z}, \mathbf{z}_\rho)$;
4: **assert** $\|\mathbf{z}\|_2 \leq \sigma\sqrt{2md}$;
5: **assert** $\|\mathbf{z}_\rho\|_2 \leq \sigma_\rho\sqrt{2rd}$;
6: **assert** $\mathbf{Az} + \mathbf{A}_\rho\mathbf{z}_\rho - c\mathbf{t}_A = \mathbf{w}$;
7: **return** $[\![\mathbf{Lz} + \mathbf{R}(c\mathbf{t}_B - \mathbf{Bz}_\rho) - c\mathbf{s} = \mathbf{v}]\!]$.

In practice, the prover sends only a hash of $(\mathbf{w}, \mathbf{v})$: the proof size does not depend on N.

LNP22: Choice of the Ring

Question #7

Check that the last verification equation holds for an honest prover.
Hint: first show that $c\mathbf{t}_B - \mathbf{B}\mathbf{z}_\rho = c\mathbf{m} - \mathbf{B}\mathbf{y}_\rho$.

LNP22: Choice of the Ring

Question #7

Check that the last verification equation holds for an honest prover.
Hint: first show that $c\mathbf{t}_B - \mathbf{B}\mathbf{z}_\rho = c\mathbf{m} - \mathbf{B}\mathbf{y}_\rho$.

LNP22 also avoid the invertibility problem of ALS20:

- $q \equiv 5 \pmod 8$: $T^d + 1$ splits into only *two* irreducible factors modulo q;

LNP22: Choice of the Ring

Question #7

Check that the last verification equation holds for an honest prover.
Hint: first show that $c\mathbf{t}_B - \mathbf{B}\mathbf{z}_\rho = c\mathbf{m} - \mathbf{B}\mathbf{y}_\rho$.

LNP22 also avoid the invertibility problem of ALS20:

- $q \equiv 5 \pmod 8$: $T^d + 1$ splits into only *two* irreducible factors modulo q;
- challenges are short polynomials fixed by the automorphism $T \mapsto T^{-1}$ (reason: in a moment);

LNP22: Choice of the Ring

Question #7

Check that the last verification equation holds for an honest prover.
Hint: first show that $c\mathbf{t}_B - \mathbf{B}\mathbf{z}_\rho = c\mathbf{m} - \mathbf{B}\mathbf{y}_\rho$.

LNP22 also avoid the invertibility problem of ALS20:

- $q \equiv 5 \pmod 8$: $T^d + 1$ splits into only *two* irreducible factors modulo q;
- challenges are short polynomials fixed by the automorphism $T \mapsto T^{-1}$ (reason: in a moment);
- every non-zero element fixed by $T \mapsto T^{-1}$ is invertible $\Rightarrow$ differences of distinct challenges are invertible, no soundness amplification needed.

From $\mathcal{R}_q$ to $\mathbb{Z}_q$: Constant Coefficients

Pack $\mathbb{Z}_q$ values into the *coefficients* of polynomials. For $f \in \mathcal{R}_q$:

- $f^* := f(T^{-1})$ is its **conjugate** ($T^{-1} = -T^{d-1}$); applied component-wise on vectors;
- $\mathrm{ct}(f) := f_0$ is its **constant coefficient**.

From $\mathcal{R}_q$ to $\mathbb{Z}_q$: Constant Coefficients

Pack $\mathbb{Z}_q$ values into the *coefficients* of polynomials. For $f \in \mathcal{R}_q$:

- $f^* := f(T^{-1})$ is its **conjugate** ($T^{-1} = -T^{d-1}$); applied component-wise on vectors;
- $\mathrm{ct}(f) := f_0$ is its **constant coefficient**.

Lemma

(i) For $f := \sum_{i<d} f_i T^i$ and $g := \sum_{i<d} g_i T^i$: $\mathrm{ct}(f^* g) = \sum_{i<d} f_i g_i$;

(ii) for $\mathbf{f}, \mathbf{g} \in \mathcal{R}_q^n$ with $f_i = \sum_{j<d} f_{i,j} T^j$, $g_i = \sum_{j<d} g_{i,j} T^j$: $\mathrm{ct}(\mathbf{f}^{*\top} \mathbf{g}) = \sum_{i,j} f_{i,j} g_{i,j}$.

From $\mathcal{R}_q$ to $\mathbb{Z}_q$: Constant Coefficients

Pack $\mathbb{Z}_q$ values into the *coefficients* of polynomials. For $f \in \mathcal{R}_q$:

- $f^* := f(T^{-1})$ is its **conjugate** ($T^{-1} = -T^{d-1}$); applied component-wise on vectors;
- $\mathrm{ct}(f) := f_0$ is its **constant coefficient**.

Lemma

(i) For $f := \sum_{i<d} f_i T^i$ and $g := \sum_{i<d} g_i T^i$: $\mathrm{ct}(f^* g) = \sum_{i<d} f_i g_i$;

(ii) for $\mathbf{f}, \mathbf{g} \in \mathcal{R}_q^n$ with $f_i = \sum_{j<d} f_{i,j} T^j$, $g_i = \sum_{j<d} g_{i,j} T^j$: $\mathrm{ct}(\mathbf{f}^{*\top} \mathbf{g}) = \sum_{i,j} f_{i,j} g_{i,j}$.

Proof. $f^* g = \sum_{i,j} f_i g_j T^{j-i}$, and for $0 \le i, j < d$, $T^{j-i} = \pm T^k$ with $k > 0$ unless $i = j$. Sum over components for *(ii)*. ■

From $\mathcal{R}_q$ to $\mathbb{Z}_q$: Constant Coefficients

Pack $\mathbb{Z}_q$ values into the *coefficients* of polynomials. For $f \in \mathcal{R}_q$:

- $f^* := f(T^{-1})$ is its **conjugate** ($T^{-1} = -T^{d-1}$); applied component-wise on vectors;
- $\mathrm{ct}(f) := f_0$ is its **constant coefficient**.

Lemma

(i) For $f := \sum_{i<d} f_i T^i$ and $g := \sum_{i<d} g_i T^i$: $\mathrm{ct}(f^* g) = \sum_{i<d} f_i g_i$;

(ii) for $\mathbf{f}, \mathbf{g} \in \mathcal{R}_q^n$ with $f_i = \sum_{j<d} f_{i,j} T^j$, $g_i = \sum_{j<d} g_{i,j} T^j$: $\mathrm{ct}(\mathbf{f}^{*\top} \mathbf{g}) = \sum_{i,j} f_{i,j} g_{i,j}$.

Proof. $f^* g = \sum_{i,j} f_i g_j T^{j-i}$, and for $0 \le i, j < d$, $T^{j-i} = \pm T^k$ with $k > 0$ unless $i = j$. Sum over components for *(ii)*. ■

Remark

In particular, $\mathrm{ct}(f^* f) = \|f\|_2^2$.

Encoding $\mathbb{Z}_q$ Statements

Example

1. To show $\sum_{i\in[d]} a_i x_i = b$ for public $a_0, \ldots, a_{d-1}, b \in \mathbb{Z}_q$: let $a := \sum_{i\in[d]} a_i T^i$, $x := \sum_{i\in[d]} x_i T^i$, and require $\mathsf{ct}(a^* x - b) = 0$.

Encoding $\mathbb{Z}_q$ Statements

Example

1. To show $\sum_{i\in[d]} a_i x_i = b$ for public $a_0, \ldots, a_{d-1}, b \in \mathbb{Z}_q$: let $a := \sum_{i\in[d]} a_i T^i$, $x := \sum_{i\in[d]} x_i T^i$, and require $\mathsf{ct}(a^* x - b) = 0$.
2. To show $x_i = 0$ for all $i > \nu$: enforce $\mathsf{ct}((T^i)^* x) = 0$ for all $i > \nu$.

Encoding $\mathbb{Z}_q$ Statements

Example

1. To show $\sum_{i\in[d]} a_i x_i = b$ for public $a_0, \ldots, a_{d-1}, b \in \mathbb{Z}_q$: let $a := \sum_{i\in[d]} a_i T^i$, $x := \sum_{i\in[d]} x_i T^i$, and require $\mathsf{ct}(a^* x - b) = 0$.
2. To show $x_i = 0$ for all $i > \nu$: enforce $\mathsf{ct}((T^i)^* x) = 0$ for all $i > \nu$.

Both have the same shape: public $\mathcal{R}_q$-affine functions $F_1, \ldots, F_M$ of the committed messages with

$$\mathsf{ct}(F_j(\mathbf{x}, \mathbf{m})) = 0 \quad \text{for all } j \in [M].$$

Encoding $\mathbb{Z}_q$ Statements

Example

1. To show $\sum_{i\in[d]} a_i x_i = b$ for public $a_0, \ldots, a_{d-1}, b \in \mathbb{Z}_q$: let $a := \sum_{i\in[d]} a_i T^i$, $x := \sum_{i\in[d]} x_i T^i$, and require $\mathsf{ct}(a^* x - b) = 0$.
2. To show $x_i = 0$ for all $i > \nu$: enforce $\mathsf{ct}((T^i)^* x) = 0$ for all $i > \nu$.

Both have the same shape: public $\mathcal{R}_q$-affine functions $F_1, \ldots, F_M$ of the committed messages with

$$\mathsf{ct}(F_j(\mathbf{x}, \mathbf{m})) = 0 \quad \text{for all } j \in [M].$$

Problem

The linear proof shows that a relation vanishes as a whole polynomial, but says nothing about individual coefficients. Revealing $F(\mathbf{x}, \mathbf{m})$ leaks the other $d - 1$ coefficients!

Proving a Constant Coefficient

Mask: a committed g with $\mathrm{ct}(g) = 0$ and other coefficients uniform. Reveal $h := g + F(\mathbf{x}, \mathbf{m})$: then $\mathrm{ct}(h) = \mathrm{ct}(F(\mathbf{x}, \mathbf{m}))$, and "$h$ is computed correctly" is an ordinary linear relation.

Proving a Constant Coefficient

Mask: a committed g with $\mathrm{ct}(g) = 0$ and other coefficients uniform. Reveal $h := g + F(\mathbf{x}, \mathbf{m})$: then $\mathrm{ct}(h) = \mathrm{ct}(F(\mathbf{x}, \mathbf{m}))$, and "$h$ is computed correctly" is an ordinary linear relation.

$\mathcal{P}(\mathsf{pp}, \mathbb{x} = \mathsf{T}, \mathbb{w} = (\mathbf{x}, \mathbf{m}, \boldsymbol{\rho}))$:

1: $g_1, \ldots, g_k \leftarrow\$ \{f \in \mathcal{R}_q : \mathrm{ct}(f) = 0\}$;
2: **send** $\mathbf{t}_g \leftarrow \mathbf{B}_g \boldsymbol{\rho} + \mathbf{g}$; *// commitment*
3: **receive** $\{\gamma_{i,j}\}_{i \in [k], j \in [M]}$;
4: $h_i \leftarrow g_i + \sum_j \gamma_{i,j} F_j(\mathbf{x}, \mathbf{m})$ for $i \in [k]$;
5: **send** $\mathbf{h} = (h_1, \ldots, h_k)$;
6: run the linear proof on $(\mathbf{t}_A, \mathbf{t}_B, \mathbf{t}_g)$ for $g_i + \sum_j \gamma_{i,j} F_j(\mathbf{x}, \mathbf{m}) = h_i$.

$\mathcal{V}(\mathsf{pp}, \mathbb{x} = (\mathsf{T}, \{F_j\}_{j \in [M]}))$:

1: **receive** $\mathbf{t}_g$;
2: **send** $\gamma_{i,j} \leftarrow\$ \mathbb{Z}_q$ for $(i,j) \in [k] \times [M]$;
3: **receive** $\mathbf{h}$;
4: **assert** $\mathrm{ct}(h_i) = 0$ for all $i \in [k]$;
5: **return** the output of the linear proof verifier.

Proving a Constant Coefficient

Mask: a committed g with $\mathrm{ct}(g) = 0$ and other coefficients uniform. Reveal $h := g + F(\mathbf{x}, \mathbf{m})$: then $\mathrm{ct}(h) = \mathrm{ct}(F(\mathbf{x}, \mathbf{m}))$, and "$h$ is computed correctly" is an ordinary linear relation.

$\mathcal{P}(\mathsf{pp}, \mathbb{x} = \mathsf{T}, \mathbb{w} = (\mathbf{x}, \mathbf{m}, \boldsymbol{\rho}))$:

1: $g_1, \ldots, g_k \leftarrow\$ \{f \in \mathcal{R}_q : \mathrm{ct}(f) = 0\}$;
2: **send** $\mathbf{t}_g \leftarrow \mathbf{B}_g\boldsymbol{\rho} + \mathbf{g}$; *// commitment*
3: **receive** $\{\gamma_{i,j}\}_{i\in[k],j\in[M]}$;
4: $h_i \leftarrow g_i + \sum_j \gamma_{i,j} F_j(\mathbf{x}, \mathbf{m})$ for $i \in [k]$;
5: **send** $\mathbf{h} = (h_1, \ldots, h_k)$;
6: run the linear proof on $(\mathbf{t}_A, \mathbf{t}_B, \mathbf{t}_g)$ for $g_i + \sum_j \gamma_{i,j} F_j(\mathbf{x}, \mathbf{m}) = h_i$.

$\mathcal{V}(\mathsf{pp}, \mathbb{x} = (\mathsf{T}, \{F_j\}_{j\in[M]}))$:

1: **receive** $\mathbf{t}_g$;
2: **send** $\gamma_{i,j} \leftarrow\$ \mathbb{Z}_q$ for $(i,j) \in [k] \times [M]$;
3: **receive** $\mathbf{h}$;
4: **assert** $\mathrm{ct}(h_i) = 0$ for all $i \in [k]$;
5: **return** the output of the linear proof verifier.

Why $k \ll M$ masks suffice. If $\mathrm{ct}(F_j) \neq 0$ for some j, then $\mathrm{ct}(h_i) = \sum_j \gamma_{i,j} \mathrm{ct}(F_j)$ is uniform over $\gamma_{i,\cdot}$ (g_i was committed before), so it vanishes w.p. $1/q$: soundness error q^{-k}.

Quadratic Relations

Collect messages and conjugates into the *extended witness* $\mathbf{x}_\varphi := \varphi(\mathbf{x}, \mathbf{m}) := (\mathbf{x}, \mathbf{x}^*, \mathbf{m}, \mathbf{m}^*)$. Goal: $f(\mathbf{x}_\varphi) = \mathbf{x}_\varphi^\top \mathbf{Q} \mathbf{x}_\varphi + \langle \mathbf{p}, \mathbf{x}_\varphi \rangle + p_0 = 0$.

Quadratic Relations

Collect messages and conjugates into the *extended witness* $\mathbf{x}_\varphi := \varphi(\mathbf{x}, \mathbf{m}) := (\mathbf{x}, \mathbf{x}^*, \mathbf{m}, \mathbf{m}^*)$. Goal: $f(\mathbf{x}_\varphi) = \mathbf{x}_\varphi^\top \mathbf{Q} \mathbf{x}_\varphi + \langle \mathbf{p}, \mathbf{x}_\varphi \rangle + p_0 = 0$.

- Taking conjugates: $\mathbf{z}^* = c^* \mathbf{x}^* + \mathbf{y}^*$, a masked opening of $\mathbf{x}^*$ with the *same* challenge only if $c^* = c$. That is why the challenges are fixed by conjugation!

Quadratic Relations

Collect messages and conjugates into the *extended witness* $\mathbf{x}_\varphi := \varphi(\mathbf{x}, \mathbf{m}) := (\mathbf{x}, \mathbf{x}^*, \mathbf{m}, \mathbf{m}^*)$. Goal: $f(\mathbf{x}_\varphi) = \mathbf{x}_\varphi^\top \mathbf{Q} \mathbf{x}_\varphi + \langle \mathbf{p}, \mathbf{x}_\varphi \rangle + p_0 = 0$.

- Taking conjugates: $\mathbf{z}^* = c^* \mathbf{x}^* + \mathbf{y}^*$, a masked opening of $\mathbf{x}^*$ with the *same* challenge only if $c^* = c$. That is why the challenges are fixed by conjugation!
- Then the verifier computes $\mathbf{z}_\varphi := \varphi(\mathbf{z}, c\mathbf{t}_B - \mathbf{B}\mathbf{z}_\rho) = c\mathbf{x}_\varphi + \mathbf{y}_\varphi$, where $\mathbf{y}_\varphi := \varphi(\mathbf{y}, -\mathbf{B}\mathbf{y}_\rho)$.

Quadratic Relations

Collect messages and conjugates into the *extended witness* $\mathbf{x}_\varphi := \varphi(\mathbf{x}, \mathbf{m}) := (\mathbf{x}, \mathbf{x}^*, \mathbf{m}, \mathbf{m}^*)$. Goal: $f(\mathbf{x}_\varphi) = \mathbf{x}_\varphi^\top \mathbf{Q} \mathbf{x}_\varphi + \langle \mathbf{p}, \mathbf{x}_\varphi \rangle + p_0 = 0$.

- Taking conjugates: $\mathbf{z}^* = c^* \mathbf{x}^* + \mathbf{y}^*$, a masked opening of $\mathbf{x}^*$ with the *same* challenge only if $c^* = c$. That is why the challenges are fixed by conjugation!
- Then the verifier computes $\mathbf{z}_\varphi := \varphi(\mathbf{z}, c\mathbf{t}_B - \mathbf{B}\mathbf{z}_\rho) = c\mathbf{x}_\varphi + \mathbf{y}_\varphi$, where $\mathbf{y}_\varphi := \varphi(\mathbf{y}, -\mathbf{B}\mathbf{y}_\rho)$.

Substituting into the quadratic function:

$$\mathbf{z}_\varphi^\top \mathbf{Q} \mathbf{z}_\varphi + c\langle \mathbf{p}, \mathbf{z}_\varphi \rangle + c^2 p_0 = f(\mathbf{x}_\varphi) c^2 + a_1 c + a_0,$$

where $a_1 := \mathbf{x}_\varphi^\top \mathbf{Q} \mathbf{y}_\varphi + \mathbf{y}_\varphi^\top \mathbf{Q} \mathbf{x}_\varphi + \langle \mathbf{p}, \mathbf{y}_\varphi \rangle$ and $a_0 := \mathbf{y}_\varphi^\top \mathbf{Q} \mathbf{y}_\varphi$.

Quadratic Relations: Garbage and Batching

As in ALS20:

1: commit to the linear coefficient: $t := \langle \mathbf{b}, \boldsymbol{\rho} \rangle + a_1$ for a public $\mathbf{b} \leftarrow\!\$\ \mathcal{R}_q^r$;

Quadratic Relations: Garbage and Batching

As in ALS20:

1: commit to the linear coefficient: $t := \langle \mathbf{b}, \boldsymbol{\rho} \rangle + a_1$ for a public $\mathbf{b} \leftarrow\!\$\ \mathcal{R}_q^r$;
2: mask the constant term: $v := \langle \mathbf{b}, \mathbf{y}_\rho \rangle + a_0$;

Quadratic Relations: Garbage and Batching

As in ALS20:

1: commit to the linear coefficient: $t := \langle \mathbf{b}, \boldsymbol{\rho} \rangle + a_1$ for a public $\mathbf{b} \leftarrow\!\$\ \mathcal{R}_q^r$;

2: mask the constant term: $v := \langle \mathbf{b}, \mathbf{y}_\rho \rangle + a_0$;

3: the verifier computes the masked opening $ct - \langle \mathbf{b}, \mathbf{z}_\rho \rangle = ca_1 - \langle \mathbf{b}, \mathbf{y}_\rho \rangle$ and checks

$$\mathbf{z}_\varphi^\top \mathbf{Q} \mathbf{z}_\varphi + c\langle \mathbf{p}, \mathbf{z}_\varphi \rangle + c^2 p_0 - (ct - \langle \mathbf{b}, \mathbf{z}_\rho \rangle) = v,$$

which holds exactly when $f(\mathbf{x}_\varphi)$ vanishes.

Quadratic Relations: Garbage and Batching

As in ALS20:

1: commit to the linear coefficient: $t := \langle \mathbf{b}, \boldsymbol{\rho} \rangle + a_1$ for a public $\mathbf{b} \leftarrow\!\$\ \mathcal{R}_q^r$;
2: mask the constant term: $v := \langle \mathbf{b}, \mathbf{y}_\rho \rangle + a_0$;
3: the verifier computes the masked opening $ct - \langle \mathbf{b}, \mathbf{z}_\rho \rangle = ca_1 - \langle \mathbf{b}, \mathbf{y}_\rho \rangle$ and checks

$$\mathbf{z}_\varphi^\top \mathbf{Q} \mathbf{z}_\varphi + c\langle \mathbf{p}, \mathbf{z}_\varphi \rangle + c^2 p_0 - (ct - \langle \mathbf{b}, \mathbf{z}_\rho \rangle) = v,$$

which holds exactly when $f(\mathbf{x}_\varphi)$ vanishes.

Batching

Many relations $f_1, \ldots, f_N$: the verifier sends random $\mu_1, \ldots, \mu_N \in \mathcal{R}_q$, and one proves $\sum_j \mu_j f_j = 0$. Constant-coefficient statements are added to the batch. The cost is essentially independent of the number of relations.

Lifting from $\mathbb{Z}_q$ to $\mathbb{Z}$

All proofs certify relations modulo q, but norms and ranges are about integers. To show $u \in [0, 2^\tau)$: let $u = \sum_{i<\tau} u_i 2^i$, the *bit polynomial* $u(T) := \sum_{i<\tau} u_i T^i$ and

$$g_1 := \sum_{i<\tau} T^i, \qquad g_2 := \sum_{i<\tau} 2^i T^i.$$

Lifting from $\mathbb{Z}_q$ to $\mathbb{Z}$

All proofs certify relations modulo q, but norms and ranges are about integers. To show $u \in [0, 2^\tau)$: let $u = \sum_{i<\tau} u_i 2^i$, the *bit polynomial* $u(T) := \sum_{i<\tau} u_i T^i$ and

$$\mathsf{g}_1 := \sum_{i<\tau} T^i, \qquad \mathsf{g}_2 := \sum_{i<\tau} 2^i T^i.$$

By the constant coefficient lemma, for integer coefficients $u_0, \ldots, u_{d-1}$:

$$\mathsf{ct}(u^*(u - \mathsf{g}_1)) = \sum_{i<\tau} u_i(u_i - 1) + \sum_{i\geq\tau} u_i^2, \qquad \mathsf{ct}(\mathsf{g}_2^* u) = \sum_{i<\tau} 2^i u_i = u(2).$$

Lifting from $\mathbb{Z}_q$ to $\mathbb{Z}$

All proofs certify relations modulo q, but norms and ranges are about integers. To show $u \in [0, 2^\tau)$: let $u = \sum_{i<\tau} u_i 2^i$, the *bit polynomial* $u(T) := \sum_{i<\tau} u_i T^i$ and

$$\mathsf{g}_1 := \sum_{i<\tau} T^i, \qquad \mathsf{g}_2 := \sum_{i<\tau} 2^i T^i.$$

By the constant coefficient lemma, for integer coefficients $u_0, \ldots, u_{d-1}$:

$$\mathsf{ct}(u^*(u - \mathsf{g}_1)) = \sum_{i<\tau} u_i(u_i - 1) + \sum_{i\geq\tau} u_i^2, \qquad \mathsf{ct}(\mathsf{g}_2^* u) = \sum_{i<\tau} 2^i u_i = u(2).$$

- Over $\mathbb{Z}$, every term of the first sum is non-negative: it vanishes iff u is a bit polynomial. One constant coefficient certifies binarity *and* vanishing above τ!

Lifting from $\mathbb{Z}_q$ to $\mathbb{Z}$

All proofs certify relations modulo q, but norms and ranges are about integers. To show $u \in [0, 2^\tau)$: let $u = \sum_{i<\tau} u_i 2^i$, the *bit polynomial* $u(T) := \sum_{i<\tau} u_i T^i$ and

$$\mathsf{g}_1 := \sum_{i<\tau} T^i, \qquad \mathsf{g}_2 := \sum_{i<\tau} 2^i T^i.$$

By the constant coefficient lemma, for integer coefficients $u_0, \ldots, u_{d-1}$:

$$\mathrm{ct}(u^*(u - \mathsf{g}_1)) = \sum_{i<\tau} u_i(u_i - 1) + \sum_{i\geq\tau} u_i^2, \qquad \mathrm{ct}(\mathsf{g}_2^* u) = \sum_{i<\tau} 2^i u_i = u(2).$$

- Over $\mathbb{Z}$, every term of the first sum is non-negative: it vanishes iff u is a bit polynomial. One constant coefficient certifies binarity *and* vanishing above τ!
- Over $\mathbb{Z}_q$, the sum may be a non-zero multiple of q. It is fine once we know that $\|\mathbf{u}\|_2$ is small enough for the sum to stay below q.

Approximate Range Proof

Let $\mathcal{B}$ be the distribution on $\{0, \pm 1\}$ with $\Pr[0] = 1/2$ and $\Pr[\pm 1] = 1/4$.

Lemma (Random projections)

Fix m and $B \leq q/41m$. Let $\mathbf{x} \in \mathbb{Z}_q^m$ with $\|\mathbf{x}\|_2 \geq B$, and let $\mathbf{y} \in \mathbb{Z}_q^{256}$ be *arbitrary*. Then, for $\mathbf{R} \leftarrow\!\$\ \mathcal{B}^{256 \times m}$,

$$\Pr\left[\|\mathbf{R}\mathbf{x} + \mathbf{y} \bmod q\|_2 < \tfrac{1}{2} B \sqrt{26}\right] < 2^{-128}.$$

Approximate Range Proof

Let $\mathcal{B}$ be the distribution on $\{0, \pm 1\}$ with $\Pr[0] = 1/2$ and $\Pr[\pm 1] = 1/4$.

Lemma (Random projections)

Fix m and $B \leq q/41m$. Let $\mathbf{x} \in \mathbb{Z}_q^m$ with $\|\mathbf{x}\|_2 \geq B$, and let $\mathbf{y} \in \mathbb{Z}_q^{256}$ be *arbitrary*. Then, for $\mathbf{R} \leftarrow\!\$\ \mathcal{B}^{256 \times m}$,

$$\Pr\left[\|\mathbf{R}\mathbf{x} + \mathbf{y} \bmod q\|_2 < \tfrac{1}{2} B \sqrt{26}\right] < 2^{-128}.$$

1: The prover commits to a mask $\mathbf{y}_B \leftarrow\!\$\ \mathcal{D}_{\sigma_B}^{256}$ in the BDLOP part;

Approximate Range Proof

Let $\mathcal{B}$ be the distribution on $\{0, \pm 1\}$ with $\Pr[0] = 1/2$ and $\Pr[\pm 1] = 1/4$.

Lemma (Random projections)

Fix m and $B \leq q/41m$. Let $\mathbf{x} \in \mathbb{Z}_q^m$ with $\|\mathbf{x}\|_2 \geq B$, and let $\mathbf{y} \in \mathbb{Z}_q^{256}$ be *arbitrary*. Then, for $\mathbf{R} \leftarrow\!\$\ \mathcal{B}^{256\times m}$,

$$\Pr\left[\|\mathbf{R}\mathbf{x} + \mathbf{y} \bmod q\|_2 < \tfrac{1}{2}B\sqrt{26}\right] < 2^{-128}.$$

1: The prover commits to a mask $\mathbf{y}_B \leftarrow\!\$\ \mathcal{D}_{\sigma_B}^{256}$ in the BDLOP part;
2: receives $\mathbf{R} \leftarrow\!\$\ \mathcal{B}^{256\times md}$ and responds with $\mathbf{z}_B := \mathbf{R}\mathbf{x} + \mathbf{y}_B$ (with rejection sampling);

Approximate Range Proof

Let $\mathcal{B}$ be the distribution on $\{0, \pm 1\}$ with $\Pr[0] = 1/2$ and $\Pr[\pm 1] = 1/4$.

Lemma (Random projections)

Fix m and $B \leq q/41m$. Let $\mathbf{x} \in \mathbb{Z}_q^m$ with $\|\mathbf{x}\|_2 \geq B$, and let $\mathbf{y} \in \mathbb{Z}_q^{256}$ be *arbitrary*. Then, for $\mathbf{R} \leftarrow\!\$\ \mathcal{B}^{256 \times m}$,

$$\Pr\left[\|\mathbf{R}\mathbf{x} + \mathbf{y} \bmod q\|_2 < \tfrac{1}{2} B \sqrt{26}\right] < 2^{-128}.$$

1: The prover commits to a mask $\mathbf{y}_B \leftarrow\!\$\ \mathcal{D}_{\sigma_B}^{256}$ in the BDLOP part;
2: receives $\mathbf{R} \leftarrow\!\$\ \mathcal{B}^{256 \times md}$ and responds with $\mathbf{z}_B := \mathbf{R}\mathbf{x} + \mathbf{y}_B$ (with rejection sampling);
3: proves with 256 constant-coefficient statements that $\mathbf{z}_B$ is correct; the verifier checks that $\|\mathbf{z}_B\|_2$ is small.

Approximate Range Proof

Let $\mathcal{B}$ be the distribution on $\{0, \pm 1\}$ with $\Pr[0] = 1/2$ and $\Pr[\pm 1] = 1/4$.

Lemma (Random projections)

Fix m and $B \leq q/41m$. Let $\mathbf{x} \in \mathbb{Z}_q^m$ with $\|\mathbf{x}\|_2 \geq B$, and let $\mathbf{y} \in \mathbb{Z}_q^{256}$ be *arbitrary*. Then, for $\mathbf{R} \leftarrow\!\$\ \mathcal{B}^{256 \times m}$,

$$\Pr\left[\|\mathbf{R}\mathbf{x} + \mathbf{y} \bmod q\|_2 < \tfrac{1}{2} B \sqrt{26}\right] < 2^{-128}.$$

1: The prover commits to a mask $\mathbf{y}_B \leftarrow\!\$\ \mathcal{D}_{\sigma_B}^{256}$ in the BDLOP part;
2: receives $\mathbf{R} \leftarrow\!\$\ \mathcal{B}^{256 \times md}$ and responds with $\mathbf{z}_B := \mathbf{R}\mathbf{x} + \mathbf{y}_B$ (with rejection sampling);
3: proves with 256 constant-coefficient statements that $\mathbf{z}_B$ is correct; the verifier checks that $\|\mathbf{z}_B\|_2$ is small.

Approximate (constant factor slack), but it rules out wraparound modulo q.

Summary

- A direct translation of Schnorr is either unsound (long secrets) or leaks the secret (short masks).

Summary

- A direct translation of Schnorr is either unsound (long secrets) or leaks the secret (short masks).
- **Rejection sampling** with discrete Gaussians makes the response independent of the secret: the Lyubashevsky protocol and signature (Fiat-Shamir with aborts).

Summary

- A direct translation of Schnorr is either unsound (long secrets) or leaks the secret (short masks).
- **Rejection sampling** with discrete Gaussians makes the response independent of the secret: the Lyubashevsky protocol and signature (Fiat-Shamir with aborts).
- Extraction is **relaxed**: longer vectors, multiplied by challenge differences.

Summary

- A direct translation of Schnorr is either unsound (long secrets) or leaks the secret (short masks).
- **Rejection sampling** with discrete Gaussians makes the response independent of the secret: the Lyubashevsky protocol and signature (Fiat-Shamir with aborts).
- Extraction is **relaxed**: longer vectors, multiplied by challenge differences.
- **BDLOP** commitments give masked openings for free; a single garbage commitment turns a product relation into a linearity check (ALS20). NTT slots pack d products of $\mathbb{Z}_q$ into one.

Summary

- A direct translation of Schnorr is either unsound (long secrets) or leaks the secret (short masks).
- **Rejection sampling** with discrete Gaussians makes the response independent of the secret: the Lyubashevsky protocol and signature (Fiat-Shamir with aborts).
- Extraction is **relaxed**: longer vectors, multiplied by challenge differences.
- **BDLOP** commitments give masked openings for free; a single garbage commitment turns a product relation into a linearity check (ALS20). NTT slots pack d products of $\mathbb{Z}_q$ into one.
- **LNP22**: ABDLOP commitments, constant coefficients via $\mathrm{ct}(f^* g) = \sum_i f_i g_i$, quadratic relations, and approximate range proofs to lift from $\mathbb{Z}_q$ to $\mathbb{Z}$.

Thank you for your attention



research.blockstream.com

github.com/BlockstreamResearch/zk-book



Blockstream